

C + + · 课 程 知 识 归 纳

C++ 课程知识归纳

语法基础 · 面向对象 · 模板与 STL

依据六份课程资料系统整理：从 C++ 基础入门到核心编程、编程提高，并汇总期末应试重点与 C/C++ 易错点，统一体例、统一版式，便于通读与复习。

初识 C++ / 数据类型 / 运算符 /
流程结构
数组 / 函数 / 指针 / 结构体

内存分区与引用 / 函数提高
类与对象 / 继承 / 多态 / 文件操
作

模板 / STL 容器 / 函数对象
常用算法 / 应试与易错点

资料范围：C++基础入门（70 页）、C++核心编程（91 页）、C++编程提高(三)（146 页）、C++期末总结超详细知识点（38 页）、C++（思维导图 1 页）、C_C++易错总结（1 页），合计 347 页。

编写体例：全书统一采用「知识清单 + 语法格式 + 代码示例 + 易错/考点提示 + 汇总表」的固定结构，章节编号连续贯通。

使用建议：第一部分打基础、第二部分抓重点、第三部分攻难点；每章末尾的提示框与第 24 章易错清单建议反复浏览。

目 录

使用说明与学习路线

- 1 本书的资料来源
- 2 全书结构
- 3 每章的固定体例
- 4 学习路线建议
- 5 关于代码与术语的约定

第 1 章 • 初识 C++

- 1 第一个C++程序
- 2 注释
- 3 变量
- 4 常量
- 5 关键字
- 6 标识符命名规则

第 2 章 • 数据类型

- 1 整型
- 2 sizeof 关键字
- 3 实型（浮点型）
- 4 字符型
- 5 转义字符
- 6 字符串型
- 7 布尔类型 bool
- 8 数据的输入

第 3 章 • 运算符

- 1 算术运算符
- 2 赋值运算符

- 3 比较运算符
- 4 逻辑运算符

第 4 章 • 程序流程结构

- 1 选择结构
- 2 循环结构
- 3 跳转语句

第 5 章 • 数组

- 1 概述
- 2 一维数组
- 3 二维数组

第 6 章 • 函数

- 1 概述
- 2 函数的定义
- 3 函数的调用
- 4 值传递
- 5 函数的常见样式
- 6 函数的声明
- 7 函数的分文件编写

第 7 章 • 指针

- 1 指针的基本概念
- 2 指针变量的定义和使用
- 3 指针所占内存空间
- 4 空指针和野指针
- 5 const 修饰指针
- 6 指针和数组
- 7 指针和函数
- 8 指针、数组、函数

第 8 章 • 结构体

- 1 结构体基本概念
- 2 结构体定义和使用
- 3 结构体数组
- 4 结构体指针
- 5 结构体嵌套结构体
- 6 结构体做函数参数
- 7 结构体中 const 使用场景
- 8 结构体案例
- 9 附：全书高频易错点 / 常考点速查

第 9 章 • 内存分区模型与 new

- 1 程序运行前
- 2 程序运行后
- 3 new 操作符

第 10 章 • 引用

- 1 引用的基本使用
- 2 引用注意事项
- 3 引用做函数参数
- 4 引用做函数返回值
- 5 引用的本质
- 6 常量引用

第 11 章 • 函数提高

- 1 函数默认参数
- 2 函数占位参数
- 3 函数重载

第 12 章 • 类与对象 • 封装

- 1 封装

第 13 章 • 类与对象 • 构造、析构与深浅拷贝

- 1 对象的初始化和清理

第 14 章 • 类与对象 • 对象模型、this 指针与友元

- 1 C++对象模型和this指针
- 2 友元

第 15 章 • 类与对象 • 运算符重载

- 1 运算符重载

第 16 章 • 继承

- 1 继承

第 17 章 • 多态

- 1 多态

第 18 章 • 文件操作

- 1 文本文件
- 2 二进制文件

第 19 章 • 模板

- 1 模板的概念
- 2 函数模板
- 3 类模板

第 20 章 • STL 初识

- 1 STL 的诞生
- 2 STL 基本概念
- 3 STL 六大组件
- 4 STL 中容器、算法、迭代器
- 5 容器、算法、迭代器初识

第 20.1 章 • STL —— string 容器

- 1 string 容器

第 20.2 章 • STL —— vector 与 deque 容器

- 1 vector 容器
- 2 deque 容器
- 3 案例 —— 评委打分

第 20.3 章 • STL —— stack、queue 与 list 容器

- 1 stack 容器
- 2 queue 容器
- 3 list 容器

第 20.4 章 • STL —— set 与 map 容器

- 1 set / multiset 容器
- 2 map / multimap 容器
- 3 案例 —— 员工分组

第 21 章 • 函数对象与谓词

- 1 函数对象
- 2 谓词
- 3 内建函数对象

第 22 章 • STL 常用算法

- 1 常用遍历算法
- 2 常用查找算法
- 3 常用排序算法
- 4 常用拷贝和替换算法
- 5 常用算术生成算法
- 6 常用集合算法
- 7 附：容器选型速查（一句话结论）

第 23 章 • 期末应试重点

- 1 本章说明
- 2 程序构成与编译过程
- 3 数据类型的高频细节
- 4 运算符与表达式
- 5 流程结构
- 6 数组
- 7 函数
- 8 指针与引用
- 9 面向对象高频细节
- 10 文件与输入输出流
- 11 考前冲刺建议

第 24 章 • C/C++ 易错点清单

- 1 本章说明
- 2 作用域运算符 `::`
- 3 指针与常量：一字之差的两种类型
- 4 常量成员函数的标准写法
- 5 不能重载的五个运算符
- 6 字符串长度与转义字符
- 7 负数的原码、反码与补码
- 8 数据类型转换
- 9 模板不是面向对象所特有的特性
- 10 二维数组的存储顺序
- 11 语法细节速记
- 12 输入输出与文件
- 13 继承相关
- 14 运算符重载的规则

- 15 类模板与模板类
- 16 静态成员与 this 指针
- 17 存储区域与变量生存期
- 18 结构体、共用体与枚举
- 19 指针运算与传参
- 20 运算符结合性与优先级
- 21 字符串函数与字符输入
- 22 典型易错题演算
- 23 一条通用检查清单

结语：复习方法与自测清单

- 1 一、三遍复习法
- 2 二、按题型准备
- 3 三、核心自测清单
- 4 四、常见失分点提醒

使用说明与学习路线

1 本书的资料来源

本书由六份 C++ 课程资料汇总整理而成，统一了章节编号、术语表述与版面样式，去除了各资料之间重复、冲突与排版噪声的部分。

序号	资料名称	规模	在本书中的作用
1	C++基础入门	70 页	第 1~8 章：语法基础（数据类型、运算符、流程结构、数组、函数、指针、结构体）
2	C++核心编程	91 页	第 9~17 章：内存分区、引用、函数提高、类与对象、继承、多态、文件操作
3	C++编程提高(三)	146 页	第 18~23 章：模板、STL 初识与常用容器、函数对象、常用算法
4	C++期末总结超详细知识点	38 页	第 24 章：期末应试重点与高频细节（覆盖约 90% 考点）
5	C++（思维导图）	1 页	面向对象与文件操作的整体知识框架，融入第 9~18、20~23 章
6	C_C++易错总结	1 页	第 25 章：易错点与易错题清单

注意

说明：第 5、6 两份资料为大幅思维导图图片，本书通过文字识别提取要点后重新组织；识别难免有个别字符误差，涉及具体结论时已按 C++ 标准语义校正。

2 全书结构

本书按「基础 → 核心 → 提高 → 应试」的顺序分三部分，共 25 章正文加一章结语：

- **第一部分 语法基础（第 1~8 章）**：程序结构、数据类型、运算符、流程控制、数组、函数、指针、结构体。这部分是后面一切内容的语法地基。
- **第二部分 面向对象（第 9~17 章）**：内存分区模型与 new、引用、函数提高（默认参数/占位参数/重载）、类与对象（封装、构造析构、深浅拷贝、初始化列表、静态成员、this 指针、友元、运算符重载）、继承、多态、文件操作。
- **第三部分 提高与应试（第 18~25 章）**：模板、STL 六大组件与常用容器、函数对象与谓词、常用算法、期末应试重点、C/C++ 易错清单。

- **结语：**复习方法、按题型的准备策略、30 条核心自测清单与常见失分点提醒。

3 每章的固定体例

全书统一采用以下五种板块，位置与样式固定，便于快速定位。

板块	作用
知识清单	用最少的文字说明「是什么、怎么写、怎么用」
语法格式	给出标准写法，可直接对照书写代码
代码示例	只保留能说明问题的最小片段，去掉了教材中的完整长程序
提示框	「注意 / 易错 / 技巧 / 考点」四类提示，集中标注踩坑位置
汇总表	把散落的规则整理成可对比、可速记的表格

4 学习路线建议

1. **第一轮（打地基，约 2 周）：**第 1 ~ 8 章通读一遍，重点把「数据类型字节数、运算符优先级、指针与数组的关系、结构体用法」弄熟。
2. **第二轮（抓重点，约 2 周）：**第 9 ~ 17 章是考试与编程能力的分水岭。务必亲手写一遍：构造函数与析构函数、深拷贝与浅拷贝、初始化列表、静态成员、运算符重载、继承构造析构顺序、多态与虚析构。
3. **第三轮（攻难点，约 2 周）：**第 18 ~ 23 章的模板与 STL 以「会用」为目标，把 vector、string、map、set 的常用接口抄写并运行一遍，理解迭代器与算法的配合方式。
4. **冲刺（考前 3 ~ 5 天）：**只看第 24 章应试重点与第 25 章易错清单，配合历年题练习。概念题与语法细节题在选择题、判断题中极易失分。

技巧

技巧：所有提示框都值得单独抄成一张 A4 纸。考前浏览自己整理的提示框，效率远高于重读正文。

5 关于代码与术语的约定

- 示例代码默认已包含 `#include <iostream>` 与 `using namespace std;`，为节省篇幅不再重复书写。
- 书中「字节数」均指 32/64 位主流平台上 `int` 为 4 字节的常见实现。
- `sizeof`、`strlen` 等返回值类型、`const` 修饰位置的区别是高频考点，凡出现处均单独标注。
- 章节内的二级编号（如「3.2 循环结构」）由本书统一自动编号，与任一原始资料的分节编号都不完全对应，引用时请以内容标题为准。

1 第一个C++程序

核心概念：编写 C++ 程序分为 4 步——创建项目、创建文件、编写代码、运行程序。

最小程序框架：

```
#include<iostream>

using namespace std;

int main() {
    cout << "Hello world" << endl;
    system("pause");
    return 0;
}
```

注意

注意：`main` 函数是程序入口，有且仅有一个；返回值类型必须写 `int`，结尾 `return 0;`。注意：`cout`、`endl` 需要 `#include<iostream>`；`using namespace std;` 用于省略 `std::` 前缀。易错：`system("pause");` 用于暂停控制台窗口，需要 `#include<cstdlib>`（VS 中常隐式可用）。

2 注释

核心概念：注释是给程序员看的说明，编译器编译时会完全忽略。

语法格式

- 单行注释：`// 描述信息` —— 通常放在一行代码上方或语句末尾。
- 多行注释：`/* 描述信息 */` —— 通常放在一段代码上方做整体说明。

易错

易错：多行注释 `/* */` 不能嵌套使用，内层 `*/` 会提前结束注释。

3 变量

核心概念：给一段指定的内存空间起名，方便操作这段内存。

语法格式：数据类型 变量名 = 初始值；

```
int a = 10;

cout << "a = " << a << endl;
```

易错

易错：C++ 中创建变量时必须给初始值，否则使用时报错/得到随机值（教材原文：不给初始值会报错）。

注意：变量名遵循标识符命名规则，不能与关键字同名。

4 常量

核心概念：记录程序中不可更改的数据。

两种定义方式

方式	语法	说明
宏常量	<code>#define 常量名 常量值</code>	通常在文件上方定义，无分号
const 常量	<code>const 数据类型 常量名 = 常量值；</code>	通常在变量定义前加 <code>const</code>

```
#define day 7

int main() {
    cout << "一周共 " << day << " 天" << endl;
    // day = 8;    // 报错：宏常量不可修改

    const int month = 12;
    // month = 24; // 报错：const 常量不可修改

    return 0;
}
```

易错

易错：`#define` 后不加分号，加分号会把分号也当成常量值的一部分。注意：两种常量都不可修改，修改即编译报错。

5 关键字

核心概念：关键字是 C++ 中预先保留的单词（标识符），定义变量或常量时不可使用。

关键字一览（C++98 常用集）

类别	关键字
类型	<code>bool</code> <code>char</code> <code>int</code> <code>float</code> <code>double</code> <code>void</code> <code>wchar_t</code> <code>short</code> <code>long</code> <code>signed</code> <code>unsigned</code>
声明/存储	<code>auto</code> <code>extern</code> <code>register</code> <code>static</code> <code>mutable</code> <code>const</code> <code>volatile</code>
流程控制	<code>if</code> <code>else</code> <code>switch</code> <code>case</code> <code>default</code> <code>for</code> <code>while</code> <code>do</code> <code>break</code> <code>continue</code> <code>goto</code> <code>return</code>
复合类型/面向对象	<code>struct</code> <code>union</code> <code>enum</code> <code>class</code> <code>public</code> <code>private</code> <code>protected</code> <code>virtual</code> <code>friend</code> <code>this</code> <code>template</code> <code>typename</code> <code>namespace</code> <code>using</code> <code>operator</code> <code>new</code> <code>delete</code> <code>explicit</code> <code>export</code> <code>typedef</code> <code>typeid</code>
异常/转换	<code>try</code> <code>catch</code> <code>throw</code> <code>static_cast</code> <code>dynamic_cast</code> <code>const_cast</code> <code>reinterpret_cast</code>
其他	<code>asm</code> <code>inline</code> <code>sizeof</code> <code>true</code> <code>false</code>

易错

易错：给变量或常量起名时使用关键字会产生歧义，直接编译失败。

6 标识符命名规则

核心概念：C++ 对变量、常量等标识符的命名有硬性规则。

4 条规则

- 标识符不能是关键字。
- 只能由字母、数字、下划线组成。
- 第一个字符必须是字母或下划线（**不能是数字**）。
- 字母区分大小写（`a` 与 `A` 是两个标识符）。

易错

易错：`2b`、`int`、`my name` 都是非法标识符。注意：建议见名知意，便于阅读。

数据类型

Data Types

核心概念：创建变量或常量时必须指定数据类型，否则无法为其分配内存。

1 整型

数据类型	占用空间	取值范围
<code>short</code> (短整型)	2 字节	$-2^{15} \sim 2^{15}-1$
<code>int</code> (整型)	4 字节	$-2^{31} \sim 2^{31}-1$
<code>long</code> (长整型)	Windows 4 字节; Linux 32 位 4 字节、64 位 8 字节	$-2^{31} \sim 2^{31}-1$
<code>long long</code> (长长整型)	8 字节	$-2^{63} \sim 2^{63}-1$

注意

注意：整型大小结论：`short < int <= long <= long long`。易错：`long` 在 Windows 与 Linux 64 位下占字节不同，跨平台代码不要假设。

2 sizeof 关键字

核心概念：统计数据类型或变量所占内存大小（字节）。

语法格式： `sizeof(数据类型 / 变量)`

```
cout << sizeof(short) << endl;    // 2
cout << sizeof(int) << endl;      // 4
cout << sizeof(long long) << endl; // 8
```

易错

易错：`sizeof` 是运算符而非函数，括号可省（对类型需加括号），且表达式不求值。常考：求数组元素个数：`sizeof(arr) / sizeof(arr[0])`。

3 实型（浮点型）

核心概念：用于表示小数，分为单精度 `float` 与双精度 `double`。

数据类型	占用空间	有效数字范围
<code>float</code>	4 字节	7 位有效数字
<code>double</code>	8 字节	15 ~ 16 位有效数字

```
float f1 = 3.14f;    // 建议加 f 后缀
double d1 = 3.14;
float f2 = 3e2;      // 科学计数法: 3 * 10^2 = 300
float f3 = 3e-2;     // 3 * 10^-2 = 0.03
```

易错

易错：默认小数常量为 `double`；用 `float` 接收 `3.14` 属于窄化，VS 会警告，建议写 `3.14f`。注意：浮点数比较不宜直接用 `==`。

4 字符型

核心概念：字符型变量用于显示单个字符，C/C++ 中只占 **1 个字节**。

语法格式： `char ch = 'a';`

```
char ch = 'a';
cout << sizeof(char) << endl; // 1
cout << (int)ch << endl;      // 97, 查看 ASCII 码
ch = 97;                      // 可直接用 ASCII 码赋值
```

易错

易错：必须用单引号括起字符，用双引号是字符串，报错。易错：单引号内只能有一个字符，`'abc'` 报错。注意：字符型并不是把字符本身放进内存，而是存放其对应的 ASCII 编码。

ASCII 码要点

字符区间	ASCII 值
控制字符 (非打印)	0 ~ 31
空格	32
数字 '0' ~ '9'	48 ~ 57
大写字母 'A' ~ 'Z'	65 ~ 90
小写字母 'a' ~ 'z'	97 ~ 122
DEL (打印字符区终点 126)	127

考点

常考: 'A' = 65, 'a' = 97, 两者相差 32; '0' = 48, 字符 '3' 转数字要 `ch - '0'`。

5 转义字符

核心概念: 用于表示一些不能直接显示出来的 ASCII 字符。

常用转义字符表

转义字符	含义	ASCII 码 (十进制)
<code>\a</code>	警报	007
<code>\b</code>	退格(BS)，将当前位置移到前一位	008
<code>\f</code>	换页(FF)，将当前位置移到下页开头	012
<code>\n</code>	换行(LF)，将当前位置移到下一行开头	010
<code>\r</code>	回车(CR)，将当前位置移到本行开头	013
<code>\t</code>	水平制表(HT)，跳到下一个 TAB 位置	009
<code>\v</code>	垂直制表(VT)	011
<code>\\</code>	代表一个反斜线字符 <code>\</code>	092
<code>\'</code>	代表一个单引号 (撇号) 字符	039
<code>\"</code>	代表一个双引号字符	034
<code>\?</code>	代表一个问号	063
<code>\0</code>	数字 0 (字符串结束符)	000
<code>\ddd</code>	8 进制转义字符，d 范围 0~7	3 位 8 进制
<code>\xhh</code>	16 进制转义字符，h 范围 0~9、a~f、A~F	3 位 16 进制

```
cout << "\\\" << endl;    // 输出一个反斜线
cout << "\tHello" << endl; // 制表符对齐
cout << "\n" << endl;     // 换行
```

易错

易错: `\t` 在中文/多行输出对齐时，只是跳到下一个 TAB 位置，不能保证严格对齐。常考：现阶段最常用：`\n`、`\\`、`\t`。

6 字符串型

核心概念：用于表示一串字符。

两种风格

1. C 风格字符串：`char 变量名[] = "字符串值";` (需 `#include<cstring>` 才能用 `strlen` 等)
2. C++ 风格字符串：`string 变量名 = "字符串值";` (需 `#include<string>`)

```
char str1[] = "hello world"; // C 风格: 双引号
string str2 = "hello world"; // C++ 风格: 需 #include<string>
cout << str1 << endl;
```

易错

易错: C 风格字符串必须用双引号括起来。易错: 字符串 "abc" 实际占 4 个字节 (a、b、c 各 1 字节 + 结尾 '\0' 1 字节)。常考: `sizeof(str)` 得到数组总字节数 (含 '\0'), `strlen(str)` 只统计到 '\0' 之前的可见字符个数。例如 `char s[]="abc";` → `sizeof(s)==4`, `strlen(s)==3` (需 `#include<cstring>`)。注意: `cin >> str` 遇到空格就停止, 含空格字符串要用 `getline(cin, str)`。

7 布尔类型 bool

核心概念: 表示真或假, 只有 `true` (本质是 1) 和 `false` (本质是 0) 两个值, 占 **1 个字节**。

```
bool flag = true;
cout << flag << endl; // 1
flag = false;
cout << flag << endl; // 0
cout << sizeof(bool) << endl; // 1
```

易错

易错: 输出 `bool` 默认打印 `1/0`, 要打印 `true/false` 需 `cout << boolalpha << flag;`。注意: 任何非 0 值赋给 `bool` 都变成 1。

8 数据的输入

核心概念: 从键盘获取数据。

语法格式: `cin >> 变量`

```
int a = 0;
cin >> a;           // 整型
double d = 0;
cin >> d;           // 浮点型
char ch = 0;
cin >> ch;          // 字符型
string str;
cin >> str;          // 字符串型（遇空格截断）
bool flag = true;
cin >> flag;         // 布尔型，输入非 0 即为 1
```

易错

易错： `cin >>` 用向右的 `>>`， `cout <<` 用向左的 `<<`，方向写反编译报错。注意：`cin` 输入字符串遇空白字符（空格、Tab、回车）即结束。

运算符

Operators

作用：用于执行代码的运算。四大类：算术、赋值、比较、逻辑。

运算符类型	作用
算术运算符	处理四则运算
赋值运算符	将表达式的值赋给变量
比较运算符	比较表达式，返回真值或假值
逻辑运算符	根据表达式的值返回真值或假值

1 算术运算符

运算符	术语	示例	结果
<code>+</code>	正号	<code>+3</code>	3
<code>-</code>	负号	<code>-3</code>	-3
<code>+</code>	加	<code>10 + 5</code>	15
<code>-</code>	减	<code>10 - 5</code>	5
<code>*</code>	乘	<code>10 * 5</code>	50
<code>/</code>	除	<code>10 / 5</code>	2
<code>%</code>	取模（取余）	<code>10 % 3</code>	1
<code>++</code>	前置递增	<code>a=2; b=++a;</code>	a=3; b=3
<code>++</code>	后置递增	<code>a=2; b=a++;</code>	a=3; b=2
<code>--</code>	前置递减	<code>a=2; b=--a;</code>	a=1; b=1
<code>--</code>	后置递减	<code>a=2; b=a--;</code>	a=1; b=2

```
int a1 = 10, b1 = 3;

cout << a1 / b1 << endl;    // 3: 两个整数相除, 结果仍是整数 (截断)

double d1 = 0.5, d2 = 0.25;

cout << d1 / d2 << endl;    // 2

int a2 = 10, b2 = ++a1 * 10;    // 前置: 先自增, 再参与表达式
int b3 = a1++ * 10;            // 后置: 先参与表达式, 再自增
```

易错

易错: 除数不能为 0, 除法与取模都不允许除数为 0。易错: 只有整型变量可以进行取模运算, 小数不能 %。易错: 两个整数相除结果仍是整数 (向下取整/截断), `10/3 == 3` 而不是 3.333。常考: 前置递增 (递减) 先对变量自增, 再计算表达式; 后置递增 (递减) 先计算表达式, 后对变量自增。

2 赋值运算符

运算符	术语	示例	结果
=	赋值	<code>a=2; b=3;</code>	<code>a=2; b=3</code>
+=	加等于	<code>a=0; a+=2;</code>	<code>a=2</code>
-=	减等于	<code>a=5; a-=3;</code>	<code>a=2</code>
=	乘等于	<code>a=2; a=2;</code>	<code>a=4</code>
/=	除等于	<code>a=4; a/=2;</code>	<code>a=2</code>
%=	模等于	<code>a=3; a%=2;</code>	<code>a=1</code>

易错

易错: `a += 2` 等价于 `a = a + 2`, 右侧是自增后的值参与运算。注意: 复合赋值运算符优先级低于算术运算符, `a *= 2 + 3` 是 `a = a * (2+3)`。

3 比较运算符

运算符	术语	示例	结果
<code>==</code>	相等	<code>4 == 3</code>	0
<code>!=</code>	不等	<code>4 != 3</code>	1
<code><</code>	小于	<code>4 < 3</code>	0
<code>></code>	大于	<code>4 > 3</code>	1
<code><=</code>	小于等于	<code>4 <= 3</code>	0
<code>>=</code>	大于等于	<code>4 >= 1</code>	1

注意

注意：C/C++ 比较运算中「真」用数字 `1` 表示，「假」用数字 `0` 表示。易错：判断相等是 `==`，写成一个 `=` 变成赋值，`if (a = b)` 逻辑错误且不报错。

4 逻辑运算符

运算符	术语	示例	结果规则
<code>!</code>	非	<code>!a</code>	a 为假则 <code>!a</code> 为真；a 为真则 <code>!a</code> 为假（真变假、假变真）
<code>&&</code>	与	<code>a && b</code>	同真为真，其余为假
<code> </code>	或	<code>a b</code>	同假为假，其余为真

```
int a = 10;
cout << !a << endl;    // 0
cout << !!a << endl;   // 1
a = 0;
int b = 0;
cout << (a && b) << endl; // 0
cout << (a || b) << endl; // 0
```

考点

常考：&& 有短路特性：左侧为假，右侧不再计算；|| 左侧为真，右侧不再计算。注意：逻辑表达式的结果也是 1（真）或 0（假）。

程序流程结构

Control Flow

三种基本结构

- 顺序结构：程序按顺序执行，不发生跳转。
- 选择结构：依据条件是否满足，有选择地执行相应功能。
- 循环结构：依据条件是否满足，循环多次执行某段代码。

1 选择结构

1.1 if 语句

三种形式

1. 单行格式：`if(条件){ 条件满足执行的语句 }`
2. 多行格式：`if(条件){ 满足执行 }else{ 不满足执行 }`
3. 多条件格式：`if(条件1){...}else if(条件2){...}...else{ 都不满足执行 }`

```
int score = 0;
cin >> score;
if (score > 600) {
    cout << "一本" << endl;
} else if (score > 500) {
    cout << "二本" << endl;
} else {
    cout << "未考上本科" << endl;
}
```

嵌套 if

```
if (score > 600) {
    if (score > 700)      cout << "北大" << endl;
    else if (score > 650) cout << "清华" << endl;
    else                 cout << "人大" << endl;
}
```

易错

易错: if 条件表达式后不要加分号, `if (a>b);` 会让后面的语句无条件执行。注意: `else` 与最近的、尚未匹配的 `if` 配对。注意: 嵌套 if 用于更精确的分层条件判断。

● 1.2 三目运算符

核心概念: 通过一个运算符实现简单判断。

语法格式: `表达式1 ? 表达式2 : 表达式3`

- 表达式1 为真 → 执行并返回表达式2 的结果;
- 表达式1 为假 → 执行并返回表达式3 的结果。

```
int a = 10, b = 20, c = 0;
c = a > b ? a : b;
cout << "c = " << c << endl;    // 20
// C++ 中三目运算符返回的是变量, 可以继续赋值
(a > b ? a : b) = 100;             // b 被赋值为 100
```

考点

常考: C++ 的三目运算符返回的是变量 (左值), 可以作为赋值左值, 这是与 C 语言的重要区别。易错: 三元运算符嵌套层级过深会导致结构不清晰, 可读性差, 此时应改用 if...else if。注意: 与 if 相比优点是短小整洁, 缺点是嵌套后不清晰。

● 1.3 switch 语句

核心概念: 执行多条件分支语句。

语法格式

```
switch (表达式)
{
    case 结果1: 执行语句; break;
    case 结果2: 执行语句; break;
    ...
    default:    执行语句; break;
}
```

```
switch (score)
{
    case 10:
    case 9: cout << "经典" << endl; break;
    case 8: cout << "非常好" << endl; break;
    default: cout << "烂片" << endl; break;
}
```

注意

注意：switch 语句中表达式类型只能是整型或者字符型（不能是浮点、字符串）。易错：case 里如果没有 `break`，程序会一直向下执行（case 穿透），多个 case 共用同一段逻辑时才会故意省略 `break`。注意：switch 不可以判断区间；判断区间要用 if。注意：与 if 相比，多条件判断时 switch 结构清晰、执行效率高。

2 循环结构

● 2.1 while 循环语句

核心概念：满足循环条件就反复执行循环语句。

语法格式： `while(循环条件){ 循环语句 }`

```
int num = 0;
while (num < 10)
{
    cout << "num = " << num << endl;
    num++;
}
```

易错

易错：循环体内必须提供跳出循环的出口（如改变条件变量），否则出现死循环。

典型应用：猜数字（需 `#include<ctime>`）

```
srand((unsigned int)time(NULL)); // 随机数种子，防止每次结果相同
int num = rand() % 100 + 1;      // 生成 1~100 的随机数
```

● 2.2 do...while 循环语句

核心概念：先执行一次循环语句，再判断循环条件。

语法格式： `do{ 循环语句 } while(循环条件);`

```
int num = 0;
do
{
    cout << num << endl;
    num++;
} while (num < 10);
```

考点

常考：与 while 的区别：do...while 会先执行一次循环语句再判断条件，循环体至少执行一次；条件初值即为假时 while 一次也不执行，而 do...while 执行一次。易错：`while(循环条件);` 末尾的分号不能丢。

典型应用：水仙花数（3 位数，各位数字的 3 次幂之和等于本身）

```
a = num % 10;           // 个位
b = num / 10 % 10;      // 十位
c = num / 100;          // 百位
if (a*a*a + b*b*b + c*c*c == num) cout << num << endl;
```

● 2.3 for 循环语句

核心概念：满足循环条件执行循环语句，结构最清晰，最常用。

语法格式： `for(起始表达式; 条件表达式; 末尾循环体){ 循环语句; }`

```
for (int i = 0; i < 10; i++)
{
    cout << i << endl;
}
```

易错

易错：for 循环中的三个表达式要用分号分隔，不是逗号。注意：三个表达式均可省略，但分号必须保留，如 `for(;;)` 为死循环。

● 2.4 嵌套循环

核心概念：在循环体中再嵌套一层循环，解决行列类问题。

典型应用：九九乘法表

```
for (int i = 1; i <= 9; i++)           // 行数
{
    for (int j = 1; j <= i; j++)       // 列数 <= 行数
    {
        cout << j << "*" << i << " = " << j * i << " ";
    }
    cout << endl;
}
```

注意

注意：外层循环执行 1 次，内层循环执行 1 轮（完整一遍）。易错：内层循环的边界通常与外层变量相关（如 `j <= i`），写死会导致图形或表格错误。

3 跳转语句

● 3.1 break 语句

核心概念：用于跳出选择结构或循环结构。

使用时机

- 出现在 switch 条件语句中：终止 case 并跳出 switch。
- 出现在循环语句中：跳出当前循环。
- 出现在嵌套循环中：跳出**最近的内层**循环。

```
for (int i = 0; i < 10; i++)
{
    if (i == 5) break;    // i 到 5 时结束整个循环
    cout << i << endl;
}
```

易错

易错：break 只能跳出最近一层循环，不能一次跳出多层嵌套循环。

● 3.2 continue 语句

核心概念：跳过本次循环中余下尚未执行的语句，继续执行下一次循环。

```
for (int i = 0; i < 100; i++)  
{  
    if (i % 2 == 0) continue;    // 跳过偶数  
    cout << i << endl;  
}
```

考点

常考：continue 并没有使整个循环终止，而 break 会跳出循环。易错：在 while/do...while 中使用 continue 时，若把自增语句放在 continue 之后，会造成死循环。

● 3.3 goto 语句

核心概念：无条件跳转语句。

语法格式： goto 标记; —— 若标记名称存在，执行到 goto 时跳转到标记处。

```
cout << "1" << endl;  
goto FLAG;  
cout << "2" << endl;    // 被跳过  
FLAG:  
cout << "5" << endl;
```

注意

注意：程序中不建议使用 goto 语句，以免造成程序流程混乱。

1 概述

核心概念：数组是一个集合，里面存放了**相同类型**的数据元素。

两大特点

1. 数组中的每个数据元素都是相同的数据类型。
2. 数组由**连续的内存位置**组成。

2 一维数组

● 2.1 一维数组定义方式

三种定义方式

1. `数据类型 数组名[数组长度];`
2. `数据类型 数组名[数组长度] = {值1, 值2, ...};`
3. `数据类型 数组名[] = {值1, 值2, ...};`

```
int score[10]; // 方式1：先定义后赋值
score[0] = 100;

int score2[10] = {100, 90, 80}; // 方式2：不足 10 个，剩余用 0 补全

int score3[] = {100, 90, 80, 70, 60}; // 方式3：自动推导长度
for (int i = 0; i < 5; i++) cout << score3[i] << endl;
```

考点

常考：数组下标从 0 开始索引，最后一个元素下标是「长度 - 1」。注意：数组名命名规范与变量名一致，不要和变量重名。易错：方式2 中若 `{}` 内数据不足，剩余元素自动补 0；方式1 不初始化则元素为随机值。

● 2.2 一维数组数组名

核心概念：数组名有两个用途——统计整个数组内存长度、获取数组首地址。

```

int arr[10] = {1,2,3,4,5,6,7,8,9,10};
cout << sizeof(arr) << endl;           // 整个数组占 40 字节
cout << sizeof(arr[0]) << endl;        // 每个元素 4 字节
cout << sizeof(arr) / sizeof(arr[0]) << endl; // 元素个数 10
cout << (int)arr << endl;               // 数组首地址
cout << (int)&arr[0] << endl;           // 第一个元素地址 (同上)
cout << (int)&arr[1] << endl;           // 第二个元素地址 (相差 4)

```

易错

易错：数组名是常量，不可以赋值，`arr = 100;` 编译报错。常考：直接打印数组名可以查看数组所占内存的首地址；对数组名 `sizeof` 得到的是整个数组占内存大小，而不是首元素指针大小。注意：相邻元素地址相差 `sizeof(元素类型)` 个字节，体现数组内存连续。

典型练习：五只小猪称体重（求最大值）、数组元素逆置（首尾对调）。

● 2.3 冒泡排序

核心概念：最常用的排序算法，对数组内元素进行排序。

算法步骤

1. 比较相邻的元素，如果第一个比第二个大，就交换他们两个。
2. 对每一对相邻元素做同样的工作，执行完毕后找到这一轮的最大值。
3. 重复以上步骤，每次比较次数 -1，直到不需要比较。

```

int arr[9] = {4,2,8,0,5,7,1,3,9};
for (int i = 0; i < 9 - 1; i++)           // 排序总轮数 = 元素个数 - 1
{
    for (int j = 0; j < 9 - 1 - i; j++) // 每轮对比次数 = 元素个数 - 轮数 - 1
    {
        if (arr[j] > arr[j + 1])
        {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}

```

考点

常考：排序总轮数 = 元素个数 - 1；每轮对比次数 = 元素个数 - 排序轮数 - 1。易错：交换必须借助临时变量，直接 `a = b; b = a;` 会丢失数据。

3 二维数组

核心概念：在一维数组上多加一个维度。

● 3.1 二维数组定义方式

四种定义方式

1. 数据类型 数组名[行数][列数];
2. 数据类型 数组名[行数][列数] = { {数据1,数据2}, {数据3,数据4} };
3. 数据类型 数组名[行数][列数] = { 数据1,数据2,数据3,数据4 };
4. 数据类型 数组名[][列数] = { 数据1,数据2,数据3,数据4 };

```
int arr2[2][3] = { {1,2,3}, {4,5,6} }; // 方式2: 最直观, 推荐
int arr3[2][3] = { 1,2,3,4,5,6 };      // 方式3
int arr4[][3] = { 1,2,3,4,5,6 };       // 方式4
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) cout << arr4[i][j] << " ";
    cout << endl;
}
```

考点

常考：定义二维数组时，可以省略行数，不能省略列数。注意：建议使用第二种（嵌套花括号）方式，可读性更好。

● 3.2 二维数组数组名

```
int arr[2][3] = { {1,2,3}, {4,5,6} };
cout << sizeof(arr) << endl;           // 整个二维数组大小 24
cout << sizeof(arr[0]) << endl;        // 一行大小 12
cout << sizeof(arr[0][0]) << endl;     // 一个元素大小 4
cout << sizeof(arr) / sizeof(arr[0]) << endl; // 行数 2
cout << sizeof(arr[0]) / sizeof(arr[0][0]) << endl; // 列数 3
cout << arr << endl;                  // 二维数组首地址
cout << arr[0] << endl;               // 第一行地址
```

注意

注意：二维数组名就是这个数组的首地址；对二维数组名 `sizeof` 可获取整个二维数组占用的内存空间大小。常考：行数 = `sizeof(arr)/sizeof(arr[0])`；列数 = `sizeof(arr[0])/sizeof(arr[0][0])`。

● 3.3 二维数组应用案例

考试成绩统计：三名同学的语数英成绩保存在 3×3 的二维数组中，分别输出每人总成绩。

```
int scores[3][3] = { {100,100,100}, {90,50,100}, {60,70,80} };
string names[3] = { "张三", "李四", "王五" };
for (int i = 0; i < 3; i++)
{
    int sum = 0;
    for (int j = 0; j < 3; j++) sum += scores[i][j];
    cout << names[i] << "同学总成绩为: " << sum << endl;
}
```

易错

易错：内层求和变量 `sum` 必须在外层循环内部清零，否则会把上一个人的成绩累加进来。

1 概述

核心概念：将一段经常使用的代码封装起来，减少重复代码；一个较大的程序一般分为若干个程序块，每个模块实现特定功能。

2 函数的定义

核心概念：函数定义一般有 5 个要素——返回值类型、函数名、参数列表、函数体语句、return 表达式。

语法格式

返回值类型 函数名 (参数列表)

```
{  
    函数体语句  
    return 表达式;  
}
```

```
int add(int num1, int num2)    // 定义加法函数  
{  
    int sum = num1 + num2;  
    return sum;  
}
```

注意

注意：返回值类型与 `return` 表达式挂钩，函数执行完返回相应数据。易错：返回值为 `void` 时不能有 `return 值;`，可写 `return;` 或省略。易错：函数不能嵌套定义，即不能在一个函数体内定义另一个函数，但可以嵌套调用。

3 函数的调用

核心概念：使用定义好的函数。

语法格式： `函数名(参数)`

```
int add(int num1, int num2) { return num1 + num2; } // 形参
int main() {
    int a = 10, b = 10;
    int sum = add(a, b); // 实参
    cout << "sum = " << sum << endl;
    return 0;
}
```

考点

常考：函数定义里小括号内的参数称为形参，函数调用时传入的参数称为实参。易错：调用时实参的个数、类型要与形参匹配。

4 值传递

核心概念：函数调用时实参将数值传入给形参。

```
void swap(int num1, int num2)
{
    int temp = num1;
    num1 = num2;
    num2 = temp;
}
int main() {
    int a = 10, b = 20;
    swap(a, b);
    cout << a << " " << b << endl; // 仍是 10 20, 未被交换
}
```

考点

常考：值传递时，形参发生改变不会影响实参，形参是修饰不了实参的。易错：用值传递写交换函数无法交换实参，必须改用地址传递（指针）。

5 函数的常见样式

4 种常见样式

1. 无参无返
2. 有参无返

3. 无参有返

4. 有参有返

```
void test01() { cout << "this is test01" << endl; } // 无参无返
void test02(int a) { cout << "a = " << a << endl; } // 有参无返
int test03() { return 10; } // 无参有返
int test04(int a, int b) { return a + b; } // 有参有返
```

易错

易错: `void` 是无类型, 不可以创建变量 (无法分配内存), `void a = 10;` 报错。注意: 函数声明为 `void` 时不需要返回值, 可以不写 `return`。

6 函数的声明

核心概念: 告诉编译器函数名称及如何调用函数, 函数的实际主体可以单独定义。

语法格式: `返回值类型 函数名(参数列表);`

```
int max(int a, int b); // 声明可以多次
int max(int a, int b);
int max(int a, int b) // 定义只能有一次
{
    return a > b ? a : b;
}
```

考点

常考: 函数的声明可以多次, 但定义只能有一次。注意: 声明的作用是解决「函数定义写在调用之后」的编译问题。

7 函数的分文件编写

核心概念: 让代码结构更加清晰。

4 个步骤

1. 创建后缀名为 `.h` 的头文件。
2. 创建后缀名为 `.cpp` 的源文件。
3. 在头文件中写函数的声明。
4. 在源文件中写函数的定义。

```
// swap.h
#include<iostream>
using namespace std;
void swap(int a, int b);           // 声明

// swap.cpp
#include "swap.h"
void swap(int a, int b) {          // 定义
    int temp = a; a = b; b = temp;
}

// main.cpp
#include "swap.h"
int main() { swap(100, 200); return 0; }
```

易错

易错：自定义头文件用双引号 `#include "swap.h"`（先搜当前目录），系统头文件用尖括号 `#include<iostream>`。注意：源文件必须包含对应的头文件，否则声明不可见。

1 指针的基本概念

核心概念：指针可以通过指针间接访问内存。

要点

- 内存编号是从 0 开始记录的，一般用十六进制数字表示。
- 可以利用指针变量保存地址。

2 指针变量的定义和使用

语法格式：数据类型 * 变量名；

```
int a = 10;
int * p;           // 定义指针
p = &a;            // 指针指向变量 a 的地址
cout << &a << endl; // 打印数据 a 的地址
cout << p << endl;   // 打印指针变量 p (同一个地址)
cout << *p << endl;  // 解引用, 输出 10
```

考点

常考：普通变量存放的是数据，指针变量存放的是地址。常考：通过 `&` 获取变量地址；通过 `*` 对指针解引用，操作指针指向的内存。易错：`int * p` 中的 `*` 是类型的一部分（声明），`*p` 是解引用运算符（使用），二者含义不同。

3 指针所占内存空间

核心概念：指针也是一种数据类型，其占用的内存空间由操作系统位数决定。

```
int a = 10;
int * p = &a;
cout << sizeof(p) << endl;           // 32 位系统: 4
cout << sizeof(char *) << endl;       // 4
cout << sizeof(float *) << endl;      // 4
cout << sizeof(double *) << endl;    // 4
```

考点

常考：所有指针类型在 32 位操作系统下都是 4 个字节（64 位下为 8 字节），与所指类型无关。

4 空指针和野指针

核心概念

- 空指针：指针变量指向内存中编号为 0 的空间。用途：初始化指针变量。
- 野指针：指针变量指向非法的内存空间。

```
int * p = NULL;           // 空指针
// cout << *p << endl;    // 报错：空指针指向的内存不可以访问
// 内存编号 0~255 为系统占用内存，不允许用户访问

int * p2 = (int *)0x1100; // 野指针
// cout << *p2 << endl;   // 报错：访问野指针
```

易错

易错：空指针指向的内存不可以访问；指针变量用完建议置为 `NULL`。注意：空指针和野指针都不是我们申请的空间，因此不要访问它们。

5 const 修饰指针

三种情况

1. `const` 修饰指针 —— 常量指针
2. `const` 修饰常量 —— 指针常量
3. `const` 既修饰指针，又修饰常量

```

int a = 10, b = 10;

// 常量指针: const 修饰的是指针, 指针指向可以改, 指向的值不可以改
const int * p1 = &a;
p1 = &b;           // 正确
// *p1 = 100;      // 报错

// 指针常量: const 修饰的是常量, 指针指向不可以改, 指向的值可以改
int * const p2 = &a;
// p2 = &b;        // 错误
*p2 = 100;         // 正确

// const 既修饰指针又修饰常量: 指向和指向的值都不可改
const int * const p3 = &a;
// p3 = &b;        // 错误
// *p3 = 100;     // 错误

```

考点

常考: 常量指针 (`const int * p`) : 值不可改, 指向可改。常考: 指针常量 (`int * const p`) : 指向不可改, 值可改。常考: `const int * const p` : 指向和值都不可改。技巧: 看 `const` 右侧紧跟着的是「指针」还是「常量」: 紧跟指针就是常量指针, 紧跟常量就是指针常量。

6 指针和数组

核心概念: 利用指针访问数组中的元素。

```

int arr[] = {1,2,3,4,5,6,7,8,9,10};
int * p = arr;           // 数组名即数组首地址, 指向数组的指针
cout << arr[0] << endl; // 第一个元素
cout << *p << endl;    // 指针访问第一个元素
for (int i = 0; i < 10; i++)
{
    cout << *p << endl;
    p++;                // 指针后移, 指向下一个元素
}

```

考点

常考：`arr == &arr[0]`，数组名就是数组首元素的地址，可直接赋给同类型指针。注意：`p++` 使指针按 `sizeof(元素类型)` 个字节向后移动，不是简单加 1。易错：指针遍历越界后不停止会访问非法内存。

7 指针和函数

核心概念：利用指针作函数参数，可以修改实参的值。

```
void swap1(int a, int b) { int t = a; a = b; b = t; }           // 值传递
void swap2(int * p1, int * p2) {                               // 地址传递
    int temp = *p1; *p1 = *p2; *p2 = temp;
}
int main() {
    int a = 10, b = 20;
    swap1(a, b);        // 值传递不会改变实参
    swap2(&a, &b);      // 地址传递会改变实参
    return 0;
}
```

考点

常考：如果不想修改实参，就用值传递；如果想修改实参，就用地址传递（传指针）。易错：地址传递时实参要加 `&`（传地址），形参要用指针类型接收。

8 指针、数组、函数

核心概念：综合运用指针 + 数组 + 函数，封装冒泡排序函数对整型数组升序排序。

```

void bubbleSort(int * arr, int len)    // int * arr 也可以写为 int arr[]
{
    for (int i = 0; i < len - 1; i++)
        for (int j = 0; j < len - 1 - i; j++)
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = temp;
            }
}

int main() {
    int arr[10] = {4,3,6,9,1,2,10,8,7,5};
    int len = sizeof(arr) / sizeof(int);
    bubbleSort(arr, len);
    return 0;
}

```

考点

常考：当数组名传入到函数作为参数时，被退化为指向首元素的指针，因此函数内 `sizeof(arr)` 得到的是指针大小（32 位下 4），必须在主函数中算好长度再传入。注意：形参 `int * arr` 与 `int arr[]` 完全等价，写法不影响语义。

1 结构体基本概念

核心概念：结构体属于**用户自定义的数据类型**，允许用户存储不同的数据类型。

2 结构体定义和使用

语法格式： `struct 结构体名 { 结构体成员列表 };`

创建结构体变量的三种方式

1. `struct 结构体名 变量名;`
2. `struct 结构体名 变量名 = { 成员1值, 成员2值, ... };`
3. 定义结构体时顺便创建变量。

```
struct student
{
    string name;    // 姓名
    int age;        // 年龄
    int score;      // 分数
} stu3;            // 方式3: 定义时顺便创建

int main() {
    struct student stu1;                // 方式1 (struct 可省略)
    stu1.name = "张三"; stu1.age = 18; stu1.score = 100;

    struct student stu2 = { "李四", 19, 60 }; // 方式2: 按成员顺序赋值
    return 0;
}
```

考点

常考：定义结构体时的关键字 `struct` 不可省略；创建结构体变量时 `struct` 可以省略（C++ 中）。注意：结构体变量利用操作符 `.` 访问成员。易错：定义结构体末尾的 `};` 分号不能丢。

3 结构体数组

核心概念：将自定义的结构体放入数组中方便维护。

语法格式： `struct 结构体名 数组名[元素个数] = { {}, {}, ... {} };`

```
struct student arr[3] =
{
    {"张三", 18, 80},
    {"李四", 19, 60},
    {"王五", 20, 70}
};
for (int i = 0; i < 3; i++)
    cout << arr[i].name << " " << arr[i].age << " " << arr[i].score << endl;
```

易错

易错：结构体数组的每个元素用花括号初始化，元素间用逗号分隔。注意：访问元素成员用 `arr[i].成员名`。

4 结构体指针

核心概念：通过指针访问结构体中的成员，利用操作符 `->` 访问结构体属性。

```
struct student stu = { "张三", 18, 100 };
struct student * p = &stu;
p->score = 80; // 指针通过 -> 操作符访问成员
cout << p->name << " " << p->age << " " << p->score << endl;
```

考点

常考：结构体指针通过 `->` 操作符访问成员，等价于 `(*p).成员名`。易错：对结构体变量本身用 `.`，对结构体指针用 `->`，混用报错。

5 结构体嵌套结构体

核心概念：结构体中的成员可以是另一个结构体。

```

struct student { string name; int age; int score; };

struct teacher
{
    int id;           // 职工编号
    string name;      // 教师姓名
    int age;          // 教师年龄
    struct student stu; // 子结构体（学生）
};

int main() {
    struct teacher t1;
    t1.id = 10000; t1.name = "老王"; t1.age = 40;
    t1.stu.name = "张三"; t1.stu.age = 18; t1.stu.score = 100;
    return 0;
}

```

注意

注意：在结构体中可以定义另一个结构体作为成员，用来解决实际问题。易错：访问嵌套成员要逐层用 `.` 展开，如 `t1.stu.name`。

6 结构体做函数参数

核心概念：将结构体作为参数向函数中传递，有两种传递方式：值传递、地址传递。

```

void printStudent(student stu)    // 值传递
{
    stu.age = 28;                // 不影响主函数中的实参
}

void printStudent2(student * stu) // 地址传递
{
    stu->age = 28;                // 会影响主函数中的实参
}

int main() {
    student stu = { "张三", 18, 100 };
    printStudent(stu);
    printStudent2(&stu);
    return 0;
}

```

考点

常考：如果不想修改主函数中的数据，用值传递；反之用地址传递。易错：地址传递要传 `&stu`，形参声明为 `student *`，函数内用 `->` 访问。

7 结构体中 const 使用场景

核心概念：用 `const` 来防止误操作。

```

void printStudent(const student * stu) // 加 const 防止函数体中的误操作
{
    // stu->age = 100;    // 操作失败，因为加了 const 修饰
    cout << "姓名: " << stu->name << " 年龄: " << stu->age << endl;
}

int main() {
    student stu = { "张三", 18, 100 };
    printStudent(&stu);
    return 0;
}

```

考点

常考：用地址传递可减少拷贝（提高效率），但函数内可能误改实参；加 `const` 修饰形参指针即可既高效又安全。易错：`const student * stu` 中不能通过 `stu->成员` 修改数据，只能读取。

8 结构体案例

● 8.1 案例1：老师带学生

需求：每名老师带 5 个学生，共 3 名老师；老师结构体含姓名和存放 5 名学生的数组；学生含姓名、分数；通过函数给老师及所带学生赋值，最终打印。

```
struct Student { string name; int score; };
struct Teacher { string name; Student sArray[5]; };

void allocateSpace(Teacher tArray[], int len)
{
    string nameSeed = "ABCDE";
    for (int i = 0; i < len; i++) {
        tArray[i].name = "教师" + nameSeed.substr(i, 1);
        for (int j = 0; j < 5; j++) {
            tArray[i].sArray[j].name = "学生" + nameSeed.substr(j, 1);
            tArray[i].sArray[j].score = rand() % 61 + 40;    // 40~100
        }
    }
}
```

注意

注意：数组做函数参数会退化为指针，需要额外传入长度 `len`。易错：`rand()` 使用前要先 `srand((unsigned int)time(NULL));`（需 `#include<ctime>`），否则每次运行结果相同。

● 8.2 案例2：英雄排序

需求：设计英雄结构体（姓名、年龄、性别），创建结构体数组存放 5 名英雄，用冒泡排序按年龄升序排序并打印。

```

struct hero { string name; int age; string sex; };

void bubbleSort(hero arr[], int len)
{
    for (int i = 0; i < len - 1; i++)
        for (int j = 0; j < len - 1 - i; j++)
            if (arr[j].age > arr[j + 1].age) {
                hero temp = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = temp;
            }
}

int main() {
    struct hero arr[5] = {
        {"刘备", 23, "男"}, {"关羽", 22, "男"}, {"张飞", 20, "男"},
        {"赵云", 21, "男"}, {"貂蝉", 19, "女"}
    };
    int len = sizeof(arr) / sizeof(hero);
    bubbleSort(arr, len);
    return 0;
}

```

考点

常考：结构体之间可以直接整体赋值（`hero temp = arr[j];`），无需逐成员拷贝，这是与数组的重要区别。注意：比较的是成员的某个字段（`arr[j].age`），交换的是整个结构体元素。

9 附：全书高频易错点 / 常考点速查

#	考点	要点
1	变量初始化	C++ 创建变量必须给初始值
2	常量	<code>#define</code> 不加分号; <code>const</code> 修饰的变量不可改
3	字符串长度	<code>"abc"</code> 占 4 字节 (含 <code>\0</code>)
4	<code>sizeof</code> <code>strlen</code> vs	<code>sizeof</code> 含 <code>\0</code> , <code>strlen</code> 不含 (需 <code>#include<cstring></code>)
5	字符引号	字符用单引号, 字符串用双引号; 单引号内只能一个字符
6	整型除法	两整数相除结果截断; 除数不能为 0; 小数不能取模
7	自增区别	前置先自增再运算; 后置先运算再自增
8	三目运算符	C++ 中返回变量 (左值), 可继续赋值; 嵌套可读性差
9	do...while	至少执行一次 循环体; <code>while(条件);</code> 分号不能丢
10	循环结构	while/do...while/for 三者可互换; for 分隔符是分号
11	break / continue	break 跳出循环, continue 只跳过本次; break 只跳最近一层
12	if 加分号	<code>if(...);</code> 使后续语句无条件执行
13	switch	表达式只能是整型或字符型; 缺 <code>break</code> 会向下穿透; 不能判区间
14	数组下标	从 0 开始, 合法范围 <code>0 ~ 长度-1</code>
15	数组名	是常量, 不能整体赋值 ; <code>sizeof(arr)</code> 得整个数组大小
16	数组初始化	<code>{}</code> 不足部分自动补 0
17	二维数组	可省略行数, 不可省略列数
18	冒泡排序	轮数 = n-1; 每轮次数 = n-轮数-1; 交换需临时变量

#	考点	要点
19	值传递	形参改变 不影响实参
20	函数嵌套	不能嵌套定义 ，但可以 嵌套调用 ；声明可多次，定义只有一次
21	数组作参数	退化为指向首元素的指针，长度必须在外边算好传入
22	指针大小	32 位系统所有指针都是 4 字节
23	空/野指针	<code>NULL</code> 与非法地址指向的内存都不可访问
24	const 修饰指针	<code>const int *p</code> 值不可改； <code>int *const p</code> 指向不可改； <code>const int *const p</code> 都不可改
25	数组与指针	<code>arr</code> 即 <code>&arr[0]</code> ； <code>p++</code> 按元素大小移动
26	结构体	定义时 <code>struct</code> 不可省，创建变量时可省；末尾 <code>;</code> 不能丢
27	结构体成员访问	变量用 <code>.</code> ，指针用 <code>-></code>
28	结构体赋值	同类型结构体可直接整体赋值（含数组成员时也可）
29	结构体做参数	值传递不影响实参，地址传递影响；不想被改就加 <code>const</code>
30	随机数	<code>srand((unsigned int)time(NULL))</code> 后 <code>rand()%n+m</code> 生成 <code>m ~ m+n-1</code>

内存分区模型与 new

Memory Model

核心概念：C++ 程序运行时把内存按用途划分为代码区、全局区、栈区、堆区四个区域；不同区域存放的数据拥有不同的生命周期，从而获得更大的编程灵活性。

区域	存放内容	分配 / 释放
代码区	函数体的二进制机器指令	操作系统管理； 共享、只读
全局区	全局变量、静态变量、常量	程序结束后由操作系统释放
栈区	函数参数值、局部变量等	编译器自动分配释放
堆区	<code>new</code> 开辟的数据	程序员手动分配释放；不释放则程序结束时由操作系统回收

1 程序运行前

核心概念：编译生成 exe 后、程序执行前，内存分为代码区与全局区两块。

- 代码区：存放 CPU 执行的机器指令；**共享**（频繁执行的程序内存中只需一份代码）；**只读**（防止程序意外修改指令）。
- 全局区：全局变量、静态变量在此；还包括常量区，字符串常量和和其他常量也在此；数据在程序结束后由操作系统释放。

```
int g_a = 10;           // 全局变量
const int c_g_a = 10;   // 全局常量
int main() {
    int a = 10;          // 局部变量：栈区
    static int s_a = 10; // 静态变量：全局区
    const int c_l_a = 10; // 局部常量：不在全局区
    cout << (int)&g_a << (int)&s_a << (int)&"hello" << endl;
}
```

- **注意：**常量区中只存放 `const` 修饰的**全局常量**和字符串常量；`const` 修饰的**局部常量**（`c_l_a`）与普通局部变量地址相邻，不在全局区。
- **易错：**全局变量与静态变量的地址相邻，字符串常量与全局 `const` 常量地址相邻——这是判断“变量在哪个区”的常用验证手段。

2 程序运行后

- **栈区**：由编译器自动分配释放，存放函数的参数值、局部变量等。
- **堆区**：由程序员分配释放；若程序员不释放，程序结束时由操作系统回收。C++ 中主要利用 `new` 在堆区开辟内存。

```
int* func() {  
    int a = 10;  
    return &a;    // 错误：返回局部变量（栈区）地址  
}  
  
int main() {  
    int* p = func();  
    cout << *p << endl; // 第一次可能打印 10（编译器保留现场）  
    cout << *p << endl; // 第二次结果为随机值  
}
```

- **易错**：不要返回局部变量的地址。栈区数据由编译器自动释放，函数结束后该地址已非法；示例中第一次打印"看似正确"只是编译器保留了现场，结果不可靠。
- **易错**：堆区数据由程序员手动开辟、手动释放，不释放只会造成内存泄漏，程序结束时才由操作系统统一回收。

3 new 操作符

核心概念： `new` 在堆区开辟数据，返回对应数据类型的指针；用 `delete` 手动释放。

语法：

```
new 数据类型                // 基本语法  
new 数据类型(初始值)        // 开辟并初始化  
new 数据类型[元素个数]      // 开辟数组  
delete 指针;                // 释放单个数据  
delete[] 指针;              // 释放数组
```

```
int* func() {  
    int* a = new int(10); // 堆区开辟, 返回 int*  
    return a;             // 可以返回: 堆区数据不会被自动释放  
}  
  
int main() {  
    int* arr = new int[10];  
    delete[] arr; // 释放数组必须加 []  
}
```

- **易错:** `delete p;` 之后该空间不可再访问 (`cout << *p;` 报错)。
- **易错:** 释放数组必须用 `delete[]`, 只写 `delete` 属于错误用法。
- **注意:** `new int(10)` 是"开辟一个 int 并初始化为 10", `new int[10]` 是"开辟 10 个 int 的数组", 括号形状不同、含义完全不同。

1 引用的基本使用

核心概念：引用给变量起别名；操作别名等同于操作原名。

语法： `数据类型 &别名 = 原名；`

```
int a = 10;
int& b = a;    // b 是 a 的别名
b = 100;       // a 也随之变为 100
```

2 引用注意事项

- **易错：**引用必须初始化，`int &c;` 是错误写法。
- **易错：**引用一旦初始化后就不可以改变绑定的对象（不能改绑）。

```
int a = 10, b = 20;
int& c = a;    // 绑定到 a，之后不可更改
c = b;         // 这是赋值操作（把 20 赋给 a），不是更改引用
```

3 引用做函数参数

核心概念：函数传参时利用引用让形参修饰实参，效果与按地址传递相同，但语法更清楚简单。

语法： `void func(int& a, int& b)`

```
void mySwap01(int a, int b) { /* 值传递：形参改变不影响实参 */ }
void mySwap02(int* a, int* b) { /* 地址传递：需解引用 */ }
void mySwap03(int& a, int& b) { int t = a; a = b; b = t; } // 引用传递
```

传参方式	形参形式	调用形式	能否修改实参
值传递	<code>int a</code>	<code>mySwap01(a, b)</code>	不能
地址传递	<code>int* a</code>	<code>mySwap02(&a, &b)</code>	能
引用传递	<code>int& a</code>	<code>mySwap03(a, b)</code>	能（且语法最简洁）

4 引用做函数返回值

核心概念：函数可以返回引用；此时函数调用可作为左值。

注意：不要返回局部变量的引用（局部变量存放在栈区，函数结束即释放）。

```
int& test02() {           // 返回静态变量引用，合法
    static int a = 20;
    return a;
}

int main() {
    int& ref = test02();
    test02() = 1000;    // 函数调用作为左值 → 必须返回引用
    cout << ref << endl; // 1000
}
```

- **易错：**返回局部变量引用（`int& test01(){ int a = 10; return a; }`）与返回局部变量地址是同一类错误。
- **注意：**只有返回引用，函数调用才可能成为左值；返回值的函数调用不能放在等号左边。

5 引用的本质

核心概念：引用的本质在 C++ 内部实现是一个**指针常量**（`int* const ref = &a;`）——指针的指向不可改，这正是"引用初始化后不能改绑"的原因。

```

void func(int& ref) {           // 编译器转换为 int* const ref = &a;
    ref = 100;                 // 转换为 *ref = 100;
}

int main() {
    int a = 10;
    int& ref = a;              // int* const ref = &a;
    ref = 20;                  // *ref = 20;
    func(a);
}

```

- **注意：**所有指针操作都由编译器自动完成，所以 C++ 推荐使用引用技术——语法方便且不会出现野指针式的书写负担。

6 常量引用

核心概念： `const` 修饰的引用，主要用来修饰形参，防止误操作修改实参。

语法： `const` 数据类型 & 引用名

```

void showValue(const int& v) { // 形参加 const 后不可修改
    // v += 10; // 报错
    cout << v << endl;
}

int main() {
    // int& ref = 10; // 错误：引用需要合法的内存空间
    const int& ref = 10; // 合法：编译器优化为 int temp = 10; const int& ref = temp;
    // ref = 100; // 错误：const 后不可修改
}

```

- **易错：** `int& ref = 10;` 错误, `const int& ref = 10;` 正确——编译器会生成临时变量承接字面量。

函数提高

Functions Revisited

1 函数默认参数

核心概念：函数形参可以有默认值，调用时未传参则使用默认值。

语法：返回值类型 函数名(参数 = 默认值) {}

```
int func(int a, int b = 10, int c = 10) { return a + b + c; }
// 1. 某位置参数有默认值后, 从该位置起向右必须都有默认值
// 2. 函数声明有默认值, 函数实现时就不能再有默认参数
int func2(int a = 10, int b = 10);
int func2(int a, int b) { return a + b; } // 实现处不能再写 =10
```

- **易错：**默认参数必须从右向左连续给出, `int func(int a = 1, int b)` 非法。
- **易错：**声明与实现不能同时给默认值 (否则报"重定义默认参数")。

2 函数占位参数

核心概念：形参列表中可以只写数据类型而不写参数名, 用于占位; 调用时必须填补该位置。

语法：返回值类型 函数名(数据类型) {}

```
void func(int a, int) { cout << "this is func" << endl; }
int main() { func(10, 10); } // 占位参数必须填补
```

- **注意：**占位参数也可以有默认值 (`void func(int a, int = 10)`) , 此时调用可不填补。
- **注意：**现阶段占位参数意义不大, 后续 (如后置 ++ 重载) 会用到该技术。

3 函数重载

3.1 函数重载概述

核心概念：函数名相同、参数不同, 提高复用性。

满足条件 (三条同时成立) :

1. 同一个作用域下

2. 函数名称相同

3. 函数参数类型不同、或个数不同、或顺序不同

- **易错**：函数的返回值不可以作为函数重载的条件。

```
void func();           // 无参
void func(int a);      // 个数/类型不同
void func(double a);   // 类型不同
void func(int a, double b);
void func(double a, int b); // 顺序不同
// int func(double a, int b) { } // 错误：仅返回值不同不构成重载
```

• 3.2 函数重载注意事项

- 引用作为重载条件：`func(int& a)` 与 `func(const int& a)` 可以共存。
- 函数重载碰到函数默认参数：可能产生二义性，需要避免。

```
void func(int& a)      { cout << "func(int&)" << endl; }
void func(const int& a) { cout << "func(const int&)" << endl; }
int main() {
    int a = 10;
    func(a); // 调用非 const 版本
    func(10); // 调用 const 版本
}
```

```
void func2(int a, int b = 10) { }
void func2(int a) { }
// func2(10); // 歧义：两个重载都能匹配，编译报错
```

- **易错**：重载 + 默认参数极易产生歧义，是最常见的编译错误来源之一。

类与对象 · 封装

Encapsulation

1 封装

1.1 封装的意义

核心概念：封装 = ①把属性和行为作为一个整体表现事物；②把属性和行为加以权限控制。

语法： `class 类名 { 访问权限：属性 / 行为 };`

```
const double PI = 3.14;
class Circle {
public:           // 访问权限：公共
    int m_r;     // 属性
    double calculateZC() { return 2 * PI * m_r; } // 行为
};
int main() {
    Circle c1;   // 用类创建对象
    c1.m_r = 10;
    cout << c1.calculateZC() << endl;
}
```

访问权限有三种：

访问权限	关键字	类内访问	类外访问	派生类（子类）访问
公共权限	<code>public</code>	可以	可以	可以
保护权限	<code>protected</code>	可以	不可以	可以
私有权限	<code>private</code>	可以	不可以	不可以

- **注意：**类内成员函数可以访问本类全部权限的属性；类外对象只能访问 `public` 成员。

1.2 struct 和 class 区别

核心概念：C++ 中 `struct` 和 `class` 唯一的区别在于默认访问权限不同。

关键字	默认访问权限
<code>struct</code>	公共 (public)
<code>class</code>	私有 (private)

```
class C1 { int m_A; }; // 默认私有: c1.m_A = 10; 错误
struct C2 { int m_A; }; // 默认公共: c2.m_A = 10; 正确
```

● 1.3 成员属性设置为私有

核心概念：把所有成员属性设为私有，对外只暴露接口。

- 优点 1：可以自己控制属性的读、写权限（可读可写 / 只读 / 只写）。
- 优点 2：对于写权限，可以检测数据的有效性。

```
class Person {
public:
    void setName(string name) { m_Name = name; } // 可读可写
    string getName() { return m_Name; }
    int getAge() { return m_Age; } // 只读
    void setAge(int age) { // 写时校验
        if (age < 0 || age > 150) { return; }
        m_Age = age;
    }
    void setLover(string lover) { m_Lover = lover; } // 只写
private:
    string m_Name; int m_Age; string m_Lover;
};
```

● 1.4 综合练习

- **练习 1（立方体类）：**设计 `Cube` 类求面积与体积；分别用**全局函数** `isSame(Cube&, Cube&)` 和**成员函数** `isSameByClass(Cube&)` 判断两个立方体是否相等。私有属性外部不可直接访问，全局函数必须通过 `public` 的 `get` 接口读取。
- **练习 2（点和圆的关系）：**设计 `Point` 与 `Circle` 两个类，`Circle` 中以 `Point m_Center` 作为成员（类对象成员）；比较 `distance` 与 `rDistance` 判断点在圆上/圆外/圆内。

类与对象 · 构造、析构与深浅拷贝

Initialization & Cleanup

1 对象的初始化和清理

1.1 构造函数和析构函数

核心概念：构造函数在创建对象时为成员属性赋值，析构函数在对象销毁前执行清理工作；两者都由编译器**自动调用**，不写则由编译器提供**空实现**。

语法：

```
构造函数： 类名() {}  
析构函数： ~类名() {}
```

构造函数特点：

1. 没有返回值，也不写 `void`
2. 函数名称与类名相同
3. 可以有参数，因此**可以发生重载**
4. 程序在调用对象时自动调用，无须手动调用，且只会调用一次

析构函数特点：

1. 没有返回值，也不写 `void`
2. 函数名称与类名相同，名称前加 `~`
3. 不可以有参数，因此**不可以发生重载**
4. 对象销毁前自动调用，无须手动调用，且只会调用一次

```
class Person {  
public:  
    Person() { cout << "Person的构造函数调用" << endl; }  
    ~Person() { cout << "Person的析构函数调用" << endl; }  
};  
  
void test01() { Person p; } // 出作用域时自动析构
```

● 1.2 构造函数的分类及调用

两种分类方式：

- 按参数分：有参构造、无参构造（无参又称默认构造函数）
- 按类型分：普通构造、拷贝构造

三种调用方式：

方式	写法	说明
括号法	<code>Person p1(10);</code>	最常用
显式法	<code>Person p2 = Person(10);</code>	<code>Person(10)</code> 单独写是匿名对象，当前行结束立即析构
隐式转换法	<code>Person p4 = 10;</code>	等价于 <code>Person p4 = Person(10);</code>

```
class Person {
public:
    Person() { } // 无参（默认）构造
    Person(int a) { age = a; } // 有参构造
    Person(const Person& p) { age = p.age; } // 拷贝构造
    ~Person() { }
    int age;
};

void test02() {
    Person p1(10); // 括号法
    Person p2 = Person(10); // 显式法
    Person p4 = 10; // 隐式转换法
}
```

- **易错**：调用无参构造函数不能加括号——`Person p2();` 会被编译器认为是**函数声明**。
- **易错**：不能用拷贝构造函数初始化匿名对象——`Person(p3);` 会被编译器当作 `Person p3;` 的对象声明。
- **注意**：`Person p3 = Person(p2);` 是拷贝构造；而 `Person p3; p3 = p2;` 是赋值操作，**不会**调用拷贝构造函数。

● 1.3 拷贝构造函数调用时机

三种时机：

1. 使用一个已经创建完毕的对象来初始化一个新对象
2. 值传递的方式给函数参数传值

3. 以值方式返回局部对象

```
void test01() {
    Person man(100);
    Person newman(man);    // 时机1: 拷贝构造
    Person newman2 = man; // 时机1: 拷贝构造
}

void doWork(Person p1) { } // 时机2: 值传递, 相当于 Person p1 = p;

Person doWork2() { Person p1; return p1; } // 时机3: 值方式返回局部对象
```

● 1.4 构造函数调用规则

默认情况下, C++ 编译器至少给一个类添加 3 个函数:

- 1. 默认构造函数 (无参, 函数体为空)
- 2. 默认析构函数 (无参, 函数体为空)
- 3. 默认拷贝构造函数, 对属性进行值拷贝

调用规则:

- 如果用户定义了**有参构造函数**, C++ 不再提供默认无参构造, 但**仍会提供默认拷贝构造**。
- 如果用户定义了**拷贝构造函数**, C++ **不会再提供其他构造函数**。
- **易错**: 一旦自己写了有参构造, `Person p1;` 就会报错 (没有默认构造可用) 。
- **易错**: 一旦自己写了拷贝构造, 无参构造和有参构造都不会再被自动提供。

● 1.5 深拷贝与浅拷贝

核心概念: 浅拷贝是简单的赋值拷贝; 深拷贝是在堆区重新申请空间后再拷贝。

对比	浅拷贝	深拷贝
实现	编译器默认拷贝构造, 逐属性赋值 <code>m_height = p.m_height</code>	自己写拷贝构造, <code>m_height = new int(*p.m_height)</code>
指针成员	两个对象指向同一块堆内存	各自拥有独立堆内存
后果	析构时同一块内存被 <code>delete</code> 两次 → 重复释放, 程序崩溃	安全

```

Person(int age, int height) {
    m_age = age;
    m_height = new int(height);    // 堆区开辟
}

Person(const Person& p) {          // 深拷贝
    m_age = p.m_age;
    m_height = new int(*p.m_height); // 重新申请空间
}

~Person() {
    if (m_height != NULL) { delete m_height; m_height = NULL; }
}

```

- **易错（面试经典）**：如果属性在堆区开辟，一定要自己提供拷贝构造函数，防止浅拷贝带来的重复释放堆区问题。
- **注意**：`delete` 后把指针置 `NULL` 是良好习惯，可避免悬空指针被再次释放。

● 1.6 初始化列表

核心概念：C++ 提供的初始化列表语法，用来初始化属性。

语法：构造函数(): 属性1(值1), 属性2(值2) ... {}

```

class Person {
public:
    // 传统方式: Person(int a,int b,int c){ m_A=a; m_B=b; m_C=c; }
    Person(int a, int b, int c) : m_A(a), m_B(b), m_C(c) {}
private:
    int m_A, m_B, m_C;
};

```

● 1.7 类对象作为类成员

核心概念：类的成员可以是另一个类的对象，该成员称为**对象成员**。当类中有对象成员时，构造顺序为**先调用对象成员的构造，再调用本类构造**；析构顺序与构造**相反**。

```

class Person {
public:
    // 初始化列表告诉编译器调用对象成员的哪一个构造函数
    Person(string name, string pName) : m_Name(name), m_Phone(pName) { }
    ~Person() { }
private:
    string m_Name;
    Phone m_Phone;    // 对象成员
};

```

- **注意：**对象成员如果只有有参构造，必须通过初始化列表显式指定其构造方式。
- **易错：**析构顺序与构造顺序相反（本类先析构，对象成员后析构）。

● 1.8 静态成员

核心概念：在成员变量和成员函数前加 `static` 关键字，称为静态成员。

静态成员变量：

- 所有对象共享同一份数据
- 在**编译阶段**分配内存
- **类内声明，类外初始化**
- 同样受访问权限限制

静态成员函数：

- 所有对象共享同一个函数
- **只能访问静态成员变量**（因为它没有 `this` 指针）
- 有两种访问方式：通过对象、通过类名
- 同样受访问权限限制

```

class Person {
public:
    static int m_A;           // 类内声明

    static void func() {
        m_A = 100;           // 可以访问静态成员变量
        // m_B = 100;       // 错误：不能访问非静态成员变量
    }

    int m_B;
private:
    static int m_B2;
};

int Person::m_A = 10;         // 类外初始化
int Person::m_B2 = 10;       // 私有静态成员也要类外初始化
// 访问: p1.m_A 或 Person::m_A; p1.func() 或 Person::func()

```

- **易错：**静态成员变量必须在**类内声明、类外初始化**（类外初始化时不能带 `static`，但要带类名作用域 `Person::`）。
- **易错：**静态成员函数不能访问非静态成员变量，也不能使用 `this` 指针。
- **注意：**静态成员变量虽在类内，但**不属于任何对象**，`sizeof(对象)` 不含它。

类与对象 · 对象模型、this 指针与友元

Object Model & this

1 C++ 对象模型和 this 指针

● 1.1 成员变量和成员函数分开存储

核心概念：类内的成员变量和成员函数**分开存储**；只有**非静态成员变量**才属于类的对象，占用对象空间。

```
class Person {  
    int mA;                // 非静态成员变量：占对象空间  
    static int mB;         // 静态成员变量：不占对象空间  
    void func() { }        // 非静态成员函数：不占对象空间  
    static void sfunc() { } // 静态成员函数：不占对象空间  
};  
// sizeof(Person) == sizeof(int)
```

- **注意：**每个非静态成员函数在内存中只有一份函数实例，多个同类型对象共用同一块代码。

● 1.2 this 指针概念

核心概念：this 指针指向被调用的成员函数所属的对象，是隐含在每一个非静态成员函数内的一种指针；不需要定义，直接使用。

this 指针的用途：

1. 当形参和成员变量**同名**时，可用 `this->` 区分
2. 在类的非静态成员函数中**返回对象本身**，可用 `return *this`

```

class Person {
public:
    Person(int age) { this->age = age; }    // 用途1: 区分同名
    Person& PersonAddPerson(Person p) {    // 用途2: 返回对象本身
        this->age += p.age;
        return *this;                    // 返回引用才能链式调用
    }
    int age;
};

// p2.PersonAddPerson(p1).PersonAddPerson(p1);

```

- **注意：**返回对象本身时函数返回类型必须写成 `Person&`（引用），否则返回的是临时拷贝，链式调用结果错误。

● 1.3 空指针访问成员函数

核心概念：C++ 中空指针也可以调用成员函数，但要注意函数体内是否用到了 `this` 指针；若用到 `this`，需要先判断以保证健壮性。

```

class Person {
public:
    void ShowClassName() { cout << "我是Person类!" << endl; }
    void ShowPerson() {
        if (this == NULL) { return; }    // 关键判断
        cout << mAge << endl;           // 访问 mAge 会使用 this
    }
    int mAge;
};

void test01() {
    Person* p = NULL;
    p->ShowClassName();    // 可以: 函数体没有用到 this
    p->ShowPerson();       // 若不加判断就会崩溃
}

```

- **易错：**成员函数中只要访问了非静态成员变量（本质是 `this->成员`），空指针调用就会崩溃。

● 1.4 const 修饰成员函数

核心概念：成员函数后加 `const` 称为**常函数**；声明对象前加 `const` 称为**常对象**。

- 常函数内不可以修改成员属性（此时 `this` 相当于 `const 类型* const`）。
- 成员属性声明时加关键字 `mutable` 后，在常函数中依然可以修改。
- 常对象只能调用常函数。

```
class Person {
public:
    void ShowPerson() const {
        // this = NULL;    // 错误: this 本质是指针常量, 指向不可改
        // m_A = 100;      // 错误: 常函数不能修改成员属性
        m_B = 100;        // 正确: mutable 修饰的成员可以修改
    }
    int m_A;
    mutable int m_B;
};

void test01() {
    const Person person;    // 常对象
    cout << person.m_A;    // 可以读
    person.m_B = 100;       // 可以改 mutable 成员
    person.ShowPerson();    // 常对象只能调用常函数
}
```

- **注意：**`this` 指针的本质是**指针常量**（`Person* const this`），指向不可改；常函数在此基础上进一步限制"指向的数据也不可改"。
- **易错：**常对象**只能**调用常函数（教材此处注释有笔误）：被 `const` 修饰的成员函数才能被常对象调用，普通成员函数不行。

2 友元

核心概念：友元让一个函数或类能够访问另一个类中的**私有成员**；关键字为 `friend`。友元的三种实现：全局函数做友元、类做友元、成员函数做友元。

● 2.1 全局函数做友元

```
class Building {  
    friend void goodGay(Building* building); // 声明在类内  
public:  
    Building() { m_SittingRoom = "客厅"; m_BedRoom = "卧室"; }  
    string m_SittingRoom;  
private:  
    string m_BedRoom;  
};  
void goodGay(Building* building) {  
    cout << building->m_SittingRoom << endl;  
    cout << building->m_BedRoom << endl; // 友元可访问私有成员  
}
```

● 2.2 类做友元

```
class Building; // 先声明类  
class goodGay {  
public:  
    goodGay();  
    void visit();  
private:  
    Building* building;  
};  
class Building {  
    friend class goodGay; // goodGay 类的所有成员函数都能访问 Building 私有成员  
public:  
    string m_SittingRoom;  
private:  
    string m_BedRoom;  
};
```

● 2.3 成员函数做友元

```
class Building;
class goodGay {
public:
    void visit();    // 只让 visit 成为好友
    void visit2();   // visit2 不能访问 Building 私有成员
private:
    Building* building;
};

class Building {
    friend void goodGay::visit();    // 成员函数做友元
private:
    string m_BedRoom;
};
```

- **注意：**成员函数做友元时，必须先声明该类（`class Building;`），再定义被友元的类，否则编译器无法识别 `goodGay::visit`。
- **注意：**`friend` 声明写在类内即可，位置不受 `public/private` 影响。

类与对象 · 运算符重载

Operator Overloading

1 运算符重载

核心概念：对已有的运算符重新进行定义，赋予其另一种功能，以适应不同的数据类型。

语法：返回值类型 operator运算符(参数列表) { ... }

运算符重载方式一览：

运算符	推荐实现方式	形式	关键点
+	成员函数或全局函数	<pre>Person operator+(const Person& p); Person operator+(const Person& p1, const Person& p2);</pre>	/ 可重载不同参数组合
<<	全局函数 + 友元	<pre>ostream& operator<<(ostream& out, Person& p);</pre>	成员函数实现不了，见下
++ 前置	成员函数	<pre>MyInteger& operator++();</pre>	先自增，再返回； 返回引用
++ 后置	成员函数	<pre>MyInteger operator++(int);</pre>	int 为占位参数；先返回旧值再自增； 返回值
=	成员函数	<pre>Person& operator=(Person& p);</pre>	堆区属性要深拷贝，返回 *this
== !=	成员函数	<pre>bool operator==(Person& p);</pre>	返回 bool
()	成员函数	<pre>void operator()(string text);</pre>	仿函数，无固定写法，非常灵活

● 1.1 加号运算符重载

作用：实现两个自定义数据类型相加。

```
class Person {
public:
    Person operator+(const Person& p) {    // 成员函数实现
        Person temp;
        temp.m_A = this->m_A + p.m_A;
        temp.m_B = this->m_B + p.m_B;
        return temp;
    }
    int m_A, m_B;
};

// 全局函数实现: Person operator+(const Person& p1, const Person& p2) { ... }
// 重载还可再重载: Person operator+(const Person& p2, int val) { ... }
// 调用: p3 = p2 + p1; 相当于 p2.operator+(p1)
//      p4 = p3 + 10; 相当于 operator+(p3, 10)
```

- **注意：**对于内置数据类型的表达式，其运算符含义不可能改变。
- **注意：**不要滥用运算符重载（避免语义混乱）。
- **注意：**运算符重载本身也可以发生函数重载。

● 1.2 左移运算符重载

作用：输出自定义数据类型。重载左移运算符配合友元可以实现输出自定义数据类型。

```
class Person {
    friend ostream& operator<<(ostream& out, Person& p);
public:
    Person(int a, int b) { this->m_A = a; this->m_B = b; }
private:
    int m_A, m_B;
};

ostream& operator<<(ostream& out, Person& p) {    // 全局函数实现
    out << "a:" << p.m_A << " b:" << p.m_B;
    return out;    // 返回 out 才能链式输出
}

// cout << p1 << "hello world" << endl;
```

- **易错**：左移运算符**不能用成员函数实现**，否则调用形式变成 `p << cout`，不是想要的效果。
- **注意**：`ostream` 对象只能有一个，必须返回 `ostream&` 才能连续输出（链式编程）。

● 1.3 递增运算符重载

作用：重载递增运算符，实现自己的整型数据。

```
class MyInteger {  
    friend ostream& operator<<(ostream& out, MyInteger myint);  
public:  
    MyInteger() { m_Num = 0; }  
    MyInteger& operator++() {           // 前置++：先++，再返回  
        m_Num++;  
        return *this;  
    }  
    MyInteger operator++(int) {         // 后置++：先返回，再++  
        MyInteger temp = *this;       // 记录当前值  
        m_Num++;  
        return temp;  
    }  
private:  
    int m_Num;  
};
```

- **易错（常考）**：前置递增返回引用，后置递增返回值。原因是前置返回自身可用引用；后置返回的是自增前的临时副本，必须按值返回。
- **注意**：后置 `++` 的 `(int)` 是占位参数，专门用于与前置 `++` 构成重载。

● 1.4 赋值运算符重载

核心概念：C++ 编译器至少给一个类添加 4 个函数：默认构造函数、默认析构函数、默认拷贝构造函数、赋值运算符 `operator=`（对属性进行值拷贝）。

易错：如果类中有属性指向堆区，做赋值操作时也会出现浅拷贝问题。

```

Person(int age) { m_Age = new int(age); }

Person& operator=(Person& p) {
    if (m_Age != NULL) {          // 先释放原有堆区内存
        delete m_Age;
        m_Age = NULL;
    }
    // m_Age = p.m_Age;           // 编译器提供的默认实现是浅拷贝
    m_Age = new int(*p.m_Age);    // 深拷贝
    return *this;                 // 返回自身, 支持连等 p3 = p2 = p1
}

```

- **易错**: 重载 `=` 时若不先 `delete` 原有堆区内存, 会造成内存泄漏; 若只做浅拷贝, 会造成重复释放。
- **注意**: 返回 `Person&` 并 `return *this`, 才能支持 `p3 = p2 = p1` 形式的连等赋值。

● 1.5 关系运算符重载

作用: 让两个自定义类型对象进行对比操作。

```

bool operator==(Person& p) {
    return this->m_Name == p.m_Name && this->m_Age == p.m_Age;
}

bool operator!=(Person& p) {
    return !(this->m_Name == p.m_Name && this->m_Age == p.m_Age);
}

```

● 1.6 函数调用运算符重载

核心概念: 函数调用运算符 `()` 也可以重载; 重载后使用方式非常像函数调用, 因此称为**仿函数**。

```

class MyAdd {
public:
    int operator()(int v1, int v2) { return v1 + v2; }
};

int main() {
    MyAdd add;
    int ret = add(10, 10);          // 像函数一样调用
    cout << MyAdd()(100, 100) << endl; // 匿名对象调用
}

```

- **注意：**仿函数没有固定写法，非常灵活。

继承

Inheritance

1 继承

核心概念：继承是面向对象三大特性之一，可以减少重复代码。`class 子类 : public 父类 {};` —— A 称为子类/派生类，B 称为父类/基类。

派生类中的成员包含两大部分：从基类继承过来的成员（表现共性）+ 自己增加的成员（体现个性）。

1.1 继承的基本语法

语法： `class 子类 : 继承方式 父类 { };`

```
class BasePage {           // 公共页面
public:
    void header() { }
    void footer() { }
    void left() { }
};

class Java : public BasePage { // 继承实现
public:
    void content() { cout << "JAVA学科视频" << endl; }
};
```

- **注意：**继承的好处是减少重复的代码；不同页面只需各自实现 `content()`。

1.2 继承方式

语法： `class 子类 : 继承方式 父类`

继承方式共三种：公共继承、保护继承、私有继承。

基类中的成员权限	<code>public</code> 公共继承	<code>protected</code> 保护继承	<code>private</code> 私有继承
<code>public</code>	<code>public</code>	<code>protected</code>	<code>private</code>
<code>protected</code>	<code>protected</code>	<code>protected</code>	<code>private</code>
<code>private</code>	不可访问	不可访问	不可访问

```
class Base1 { public: int m_A; protected: int m_B; private: int m_C; };

class Son1 : public Base1 { void func() { m_A; m_B; /*m_C 不可访问*/ } };
class Son2 : protected Base2 { void func() { m_A; m_B; /* 均为 protected */ } };
class Son3 : private Base3 { void func() { m_A; m_B; /* 均为 private */ } };
// class GrandSon3 : public Son3 → 无法访问 m_A、m_B、m_C
```

- **易错：**父类的私有成员（`private`）无论哪种继承方式，子类都**不可访问**。
- **易错：**私有继承后，再往下继承的孙子类无法访问这些成员（`Son3` 中是 `private`，`GrandSon3` 访问不到）。

● 1.3 继承中的对象模型

核心概念：从父类继承过来的成员（包括私有成员）**都属于子类对象**；父类私有成员只是被编译器隐藏、访问不到，但确实被继承下去了。

```
class Base { public: int m_A; protected: int m_B; private: int m_C; };
class Son : public Base { public: int m_D; };
// sizeof(Son) == 16 (4 ↑ int: m_A、m_B、m_C、m_D)
```

- **注意：**可用 VS 开发者工具查看对象模型：`cl /d1 reportSingleClassLayout`类名 所属文件名。

● 1.4 继承中构造和析构顺序

核心概念：先调用父类构造函数，再调用子类构造函数；**析构顺序与构造相反**（先子类析构，再父类析构）。

```
class Base { public: Base() { cout << "Base构造函数!" << endl; }
             ~Base() { cout << "Base析构函数!" << endl; } };
class Son : public Base { public: Son() { cout << "Son构造函数!" << endl; }
                          ~Son() { cout << "Son析构函数!" << endl; } };
// 输出: Base 构造 → Son 构造 → Son 析构 → Base 析构
```

● 1.5 继承同名成员处理方式

核心概念：子类对象访问子类同名成员直接访问；访问父类同名成员需要加**作用域**。

```

class Base { public: int m_A; void func(); void func(int a); };
class Son : public Base { public: int m_A; void func(); };

void test01() {
    Son s;

    cout << s.m_A << endl;           // 子类同名成员: 直接访问
    cout << s.Base::m_A << endl;      // 父类同名成员: 加作用域
    s.func();                         // 子类版本
    s.Base::func();                   // 父类版本 (无参)
    s.Base::func(10);                 // 父类版本 (有参)
}

```

- **易错**：当子类与父类拥有同名的成员函数时，子类会隐藏父类中所有版本的同名成员函数（包括重载版本）；想访问被隐藏的版本必须加父类作用域。

● 1.6 继承同名静态成员处理方式

核心概念：静态成员与非静态成员同名时的处理方式一致——子类直接访问，父类加作用域；区别是静态成员有两种访问方式（通过对象、通过类名）。

```

int Base::m_A = 100;
int Son::m_A = 200;
void test02() {
    Son s;

    cout << s.m_A << endl;           // 通过对象访问子类
    cout << s.Base::m_A << endl;      // 通过对象访问父类
    cout << Son::m_A << endl;         // 通过类名访问子类
    cout << Son::Base::m_A << endl;    // 通过类名访问父类
    Son::func(); Son::Base::func(); Son::Base::func(100);
}

```

- **易错**：通过类名访问父类静态成员要写 `Son::Base::m_A`（两个作用域），不能只写一个。

● 1.7 多继承语法

核心概念：C++ 允许一个类继承多个类。

语法： `class 子类 : 继承方式 父类1, 继承方式 父类2 ...`

```
class Son : public Base2, public Base1 {
public:
    Son() { m_C = 300; m_D = 400; }
};
// s.Base1::m_A; s.Base2::m_A; 多继承同名成员要用作用域区分
```

- **易错**：多继承容易产生成员同名（二义性），必须用类名作用域区分调用哪个基类的成员。
- **注意**：C++ 实际开发中**不建议**使用多继承。

• 1.8 菱形继承

核心概念：两个派生类继承同一个基类，又有某个类同时继承这两个派生类，称为**菱形继承（钻石继承）**。

菱形继承的两个问题：

1. **二义性**：SheepTuo 使用数据时无法确定用 Sheep 还是 Tuo 的那份。
2. **数据冗余**：公共基类的数据被继承了两份，资源浪费且毫无意义。

```
class Animal { public: int m_Age; };
class Sheep : virtual public Animal { }; // 虚继承
class Tuo : virtual public Animal { }; // 虚继承
class SheepTuo : public Sheep, public Tuo { };

void test01() {
    SheepTuo st;
    st.Sheep::m_Age = 100;
    st.Tuo::m_Age = 200; // 与上一行操作的是同一份数据
    cout << st.m_Age << endl; // 可直接访问，无二义性
}
```

- **注意**：利用**虚继承**（继承前加 virtual 关键字）可以解决菱形继承问题；此时公共父类 Animal 称为**虚基类**。
- **易错**：虚继承后 Sheep::m_Age 与 Tuo::m_Age 指向同一份数据，先写 100 再写 200，最终值为 200。

1 多态

1.1 多态的基本概念

核心概念：多态是 C++ 面向对象三大特性之一，分为静态多态与动态多态。

类型	实现方式	绑定时机
静态多态	函数重载、运算符重载（复用函数名）	地址早绑定， 编译阶段 确定函数地址
动态多态	派生类 + 虚函数	地址晚绑定， 运行阶段 确定函数地址

多态满足条件：

1. 有继承关系
2. 子类**重写**父类中的虚函数
3. （使用条件）父类指针或引用指向子类对象

重写的定义：函数返回值类型、函数名、参数列表**完全一致**。

```
class Animal {
public:
    virtual void speak() { cout << "动物在说话" << endl; } // 虚函数
};

class Cat : public Animal { public: void speak() { cout << "小猫在说话" << endl; } };
class Dog : public Animal { public: void speak() { cout << "小狗在说话" << endl; } };

void DoSpeak(Animal& animal) { animal.speak(); } // 父类引用指向子类对象
void test01() { Cat cat; DoSpeak(cat); Dog dog; DoSpeak(dog); }
```

- **注意：**函数前加 `virtual` 变成虚函数后，编译器在编译时就不能确定函数调用，从而实现动态联编。
- **易错：**若不写 `virtual`，`DoSpeak` 中永远调用 `Animal::speak`，多态失效。

1.2 多态案例一 —— 计算器类

核心概念：分别用普通写法和多态技术实现两个操作数的运算计算器，体会多态的扩展性优势。

// 普通实现: 用 if-else 判断运算符, 新增运算必须修改源码

```
int getResult(string oper) {  
    if (oper == "+") return m_Num1 + m_Num2;  
    else if (oper == "-") return m_Num1 - m_Num2;  
}
```

// 多态实现

```
class AbstractCalculator {  
public:  
    virtual int getResult() { return 0; }  
    int m_Num1, m_Num2;  
};  
class AddCalculator : public AbstractCalculator {  
public: int getResult() { return m_Num1 + m_Num2; }  
};  
void test02() {  
    AbstractCalculator* abc = new AddCalculator; // 父类指针指向子类对象  
    abc->m_Num1 = 10; abc->m_Num2 = 10;  
    cout << abc->getResult() << endl;  
    delete abc; // 用完记得销毁  
}
```

- **注意:** 多态的优点是代码组织结构清晰、可读性强、利于前期和后期的扩展与维护 (新增运算只需新增子类, 无需改源码)。
- **易错:** `new` 出来的对象必须 `delete`, 否则内存泄漏。

• 1.3 纯虚函数和抽象类

核心概念: 多态中父类虚函数的实现通常毫无意义 (都是调用子类重写内容), 因此可将虚函数改为**纯虚函数**; 含有纯虚函数的类称为**抽象类**。

语法: `virtual 返回值类型 函数名(参数列表) = 0;`

抽象类特点:

- **无法实例化对象**
- 子类必须重写抽象类中的纯虚函数, 否则该子类也属于抽象类

```

class Base {
public:
    virtual void func() = 0;    // 纯虚函数
};

class Son : public Base {
public:
    virtual void func() { cout << "func调用" << endl; }
};

void test01() {
    Base* base = NULL;
    // base = new Base;          // 错误: 抽象类无法实例化对象
    base = new Son;             // 正确: 父类指针指向子类对象
    base->func();
    delete base;
}

```

- **易错**: 抽象类不能实例化; 子类不重写纯虚函数, 子类仍是抽象类, 也不能实例化。

● 1.4 多态案例二 —— 制作饮品

核心概念: 制作饮品的流程为 煮水 → 冲泡 → 倒入杯中 → 加入辅料; 用抽象基类规定流程骨架, 子类实现各步骤。

```

class AbstractDrinking {
public:
    virtual void Boil() = 0;          // 烧水
    virtual void Brew() = 0;         // 冲泡
    virtual void PourInCup() = 0;    // 倒入杯中
    virtual void PutSomething() = 0; // 加入辅料
    void MakeDrink() {               // 规定流程 (模板方法)
        Boil(); Brew(); PourInCup(); PutSomething();
    }
};

void DoWork(AbstractDrinking* drink) {
    drink->MakeDrink();
    delete drink;
}

// DoWork(new Coffee); DoWork(new Tea);

```

● 1.5 虚析构和纯虚析构

核心概念：多态使用时，如果子类中有属性开辟到堆区，父类指针在释放时无法调用到子类的析构代码，会造成内存泄漏；解决方式是把父类析构函数改为**虚析构**或**纯虚析构**。

语法：

```
virtual ~类名() {}           // 虚析构
virtual ~类名() = 0;         // 纯虚析构（类内声明）
类名::~~类名() {}           // 纯虚析构必须在类外给出实现
```

对比	虚析构	纯虚析构
共性	都能解决"父类指针释放子类对象"的问题； 都需要有具体的函数实现	同左
区别	所在类可以作为普通类使用	所在类属于 抽象类 ，无法实例化对象

```
class Animal {
public:
    virtual void Speak() = 0;
    virtual ~Animal() = 0;           // 纯虚析构
};
Animal::~~Animal() { cout << "Animal 纯虚析构函数调用！" << endl; }

class Cat : public Animal {
public:
    Cat(string name) { m_Name = new string(name); }    // 子类堆区数据
    ~Cat() { if (m_Name != NULL) { delete m_Name; m_Name = NULL; } }
    string* m_Name;
};

void test01() {
    Animal* animal = new Cat("Tom");
    animal->Speak();
    delete animal;    // 有了虚析构，才会调用 Cat::~~Cat
}
```

- **易错（常考）：**若父类析构不是虚函数，`delete animal;` 只会调用父类析构，子类堆区数据不会被释放 → 内存泄漏。

- **注意：**如果子类中没有堆区数据，可以不写虚析构或纯虚析构。
- **注意：**虚析构和纯虚析构都需要有函数实现；拥有纯虚析构函数的类也属于抽象类。

● 1.6 多态案例三 —— 电脑组装

核心概念：把 CPU、显卡、内存条各自封装为抽象基类，不同厂商（Intel、Lenovo）继承实现；`Computer` 类通过父类指针组合三个零件，调用统一接口工作，并在析构中释放零件。

```
class CPU      { public: virtual void calculate() = 0; }; // 抽象零件
class VideoCard { public: virtual void display()   = 0; };
class Memory   { public: virtual void storage()    = 0; };

class Computer {
public:
    Computer(CPU* cpu, VideoCard* vc, Memory* mem) {
        m_cpu = cpu; m_vc = vc; m_mem = mem;
    }
    void work() { m_cpu->calculate(); m_vc->display(); m_mem->storage(); }
    ~Computer() { // 析构释放三个零件
        if (m_cpu != NULL) { delete m_cpu; m_cpu = NULL; }
        if (m_vc  != NULL) { delete m_vc;  m_vc  = NULL; }
        if (m_mem != NULL) { delete m_mem; m_mem = NULL; }
    }
private:
    CPU* m_cpu; VideoCard* m_vc; Memory* m_mem;
};
```

- **注意：**`Computer` 的构造接收的是**抽象基类指针**，因此可以传入任意厂商的具体零件，实现“零件可插拔”。

文件操作

File I/O

核心概念：程序运行时产生的数据都属于临时数据，程序结束即释放；通过文件可以将数据**持久化**。

- 头文件： `#include <fstream>`
- 文件类型：
 - **文本文件** —— 以文本的 ASCII 码形式存储在计算机中
 - **二进制文件** —— 以二进制形式存储，用户一般不能直接读懂
- 操作文件的三大类：

类名	作用
<code>ofstream</code>	写操作
<code>ifstream</code>	读操作
<code>fstream</code>	读写操作

文件打开方式：

打开方式	解释
<code>ios::in</code>	为读文件而打开文件
<code>ios::out</code>	为写文件而打开文件
<code>ios::ate</code>	初始位置：文件尾
<code>ios::app</code>	追加方式写文件
<code>ios::trunc</code>	如果文件存在先删除，再创建
<code>ios::binary</code>	二进制方式

- **注意：**文件打开方式可以配合使用，利用 `|` 操作符，如 `ios::binary | ios::out`。

1 文本文件

• 1.1 写文件

核心概念：用输出流对象 `ofstream`（或 `fstream`）配合 `<<` 向文件写数据。

写文件步骤：

1. 包含头文件 `#include <fstream>`
2. 创建流对象 `ofstream ofs;`
3. 打开文件 `ofs.open("文件路径", 打开方式);`
4. 写数据 `ofs << "写入的数据";`
5. 关闭文件 `ofs.close();`

```
#include <fstream>

void test01() {
    ofstream ofs;
    ofs.open("test.txt", ios::out);
    ofs << "姓名: 张三" << endl;
    ofs << "性别: 男" << endl;
    ofs << "年龄: 18" << endl;
    ofs.close();           // 操作完毕必须关闭文件
}
```

- **注意：**打开文件时需要指定操作文件的路径以及打开方式。
- **注意：**`ios::out` 默认会截断（trunc）已有文件内容；要追加应使用 `ios::app`。

● 1.2 读文件

核心概念：读文件步骤与写文件相似，但读取方式较多；必须判断文件是否打开成功。

读文件步骤：

1. 包含头文件 `#include <fstream>`
2. 创建流对象 `ifstream ifs;`
3. 打开文件并判断是否打开成功 `ifs.open("文件路径", 打开方式);`
4. 读数据（四种方式）
5. 关闭文件 `ifs.close();`

四种读取方式：

方式	写法	特点
1	<code>while (ifs >> buf)</code>	按空白（空格/换行）分隔读取到 <code>char buf[]</code>
2	<code>while (ifs.getline(buf, sizeof(buf)))</code>	按行读取到字符数组
3	<code>while (getline(ifs, buf))</code>	全局 <code>getline</code> ，配合 <code>string</code> 使用
4	<code>while ((c = ifs.get()) != EOF)</code>	逐字符读取，直到文件结束

```

#include <fstream>
#include <string>
void test01() {
    ifstream ifs;
    ifs.open("test.txt", ios::in);
    if (!ifs.is_open()) {
        cout << "文件打开失败" << endl;
        return;
    }
    string buf;
    while (getline(ifs, buf)) { cout << buf << endl; }
    ifs.close();
}

```

- **易错**：读文件前必须用 `is_open()` 判断是否打开成功，否则文件不存在时读取行为不确定。
- **注意**：读文件可以利用 `ifstream`，或者 `fstream` 类；写文件利用 `ofstream`，或者 `fstream` 类。
- **注意**：操作完毕都要 `close()` 关闭文件。

2 二进制文件

核心概念：以二进制的方式对文件进行读写操作，打开方式必须指定为 `ios::binary`。

● 2.1 写文件

核心概念：二进制方式写文件主要利用流对象调用成员函数 `write`。

函数原型：`ostream& write(const char* buffer, int len);` 参数解释：`buffer` 指向内存中一段存储空间，`len` 是读写的字节数。

```

#include <fstream>
class Person { public: char m_Name[64]; int m_Age; };
void test01() {
    ofstream ofs("person.txt", ios::out | ios::binary); // 构造时即可打开
    // ofs.open("person.txt", ios::out | ios::binary); // 或单独 open
    Person p = {"张三", 18};
    ofs.write((const char*)&p, sizeof(p)); // 按字节写入
    ofs.close();
}

```

- **注意**：写对象时要强制转换成 `const char*`，并用 `sizeof(p)` 给出字节数。

- **注意：**文件输出流对象可以通过 `write` 函数以二进制方式写数据。

• 2.2 读文件

核心概念：二进制方式读文件主要利用流对象调用成员函数 `read`。

函数原型： `istream& read(char* buffer, int len);` 参数解释：`buffer` 指向内存中一段存储空间，`len` 是读写的字节数。

```
#include <fstream>

void test01() {
    ifstream ifs("person.txt", ios::in | ios::binary);
    if (!ifs.is_open()) { cout << "文件打开失败" << endl; return; }

    Person p;
    ifs.read((char*)&p, sizeof(p));
    cout << "姓名: " << p.m_Name << " 年龄: " << p.m_Age << endl;
    ifs.close();
}
```

- **注意：**文件输入流对象可以通过 `read` 函数以二进制方式读数据；读写用的结构体/类布局必须一致，否则数据错位。

1 模板的概念

- **核心**：模板就是建立通用代码的“模具”，把数据类型参数化，从而大幅提高代码复用性；C++ 泛型编程所依赖的主要技术就是模板。
- **两种机制**：函数模板、类模板。
- **注意**：模板本身不是代码，只是一个框架，**不可以直接使用**。
- **注意**：模板的通用**并不是万能的**（对数组做赋值、对自定义类型做比较等都会失效，需借助模板具体化解决）。

2 函数模板

2.1 函数模板语法

- **核心**：建立一个通用函数，其返回值类型和形参类型不具体制定，用一个虚拟的类型来代表。
- **语法**：

```
template<typename T>    // typename 可以用 class 代替  
函数声明或定义
```

```
template<typename T>  
void mySwap(T& a, T& b) { T temp = a; a = b; b = temp; }  
  
mySwap(a, b);           // 方式一：自动类型推导  
mySwap<int>(a, b);      // 方式二：显式指定类型
```

- **关键词**：`template` —— 声明创建模板；`typename` / `class` —— 表明其后的符号是一种数据类型；`T` —— 通用数据类型，名称可替换，通常用大写字母。
- **规则**：使用函数模板有两种方式 —— **自动类型推导**、**显式指定类型**；模板的目的是提高复用性、将类型参数化。

2.2 函数模板注意事项

- **易错**：自动类型推导**必须推导出一致的数据类型 T**，才可以使用。`mySwap(a, c)`（int 与 char 混用）编译失败。

- **易错**：模板 **必须确定出 T 的数据类型** 才可以使用。当 T 不出现在形参中时（如 `template<class T> void func(){}），不能写 func();，必须写 func<int>();。`

● 2.3 函数模板案例（通用数组排序）

- **核心**：用函数模板封装排序函数，可对不同数据类型数组排序（规则从大到小，算法为选择排序），用 char 数组和 int 数组测试。
- **要点代码**：

```
template<class T>
void mySort(T arr[], int len) {
    for (int i = 0; i < len; i++) {
        int max = i;
        for (int j = i + 1; j < len; j++)
            if (arr[max] < arr[j]) max = j;
        if (max != i) mySwap(arr[max], arr[i]);
    }
}
```

- **规则**：数组长度用 `sizeof(arr) / sizeof(arr[0])` 求得并作为参数传入；打印同样可写成 `printArray` 模板。

● 2.4 普通函数与函数模板的区别

- **核心**：普通函数调用时可以发生自动类型转换（隐式类型转换）；函数模板在**自动类型推导**时不会发生隐式类型转换，**显式指定类型**时可以。
- **易错**：`myAdd02(a, c);`（a 为 int, c 为 char）报错；改为 `myAdd02<int>(a, c);` 正确。
- **建议**：调用函数模板时推荐使用**显式指定类型**的方式，因为可以自己确定通用类型 T。

● 2.5 普通函数与函数模板的调用规则

- **核心**：四条调用规则 ——
 - 函数模板和普通函数都能实现时，**优先调用普通函数**；
 - 可用**空模板参数列表** `myPrint<>(a, b);` 强制调用函数模板；
 - 函数模板**也可以发生重载**；
 - 如果函数模板能产生**更好的匹配**，优先调用函数模板（如传入 char 实参时精确匹配模板）。
- **易错**：如果只告诉编译器普通函数存在（只有声明没有实现，或实现不在当前文件），编译器仍会优先选择普通函数，导致**链接/编译找不到实现而报错**。
- **注意**：既然提供了函数模板，最好就不要再提供同名普通函数，否则容易出现**二义性**。

● 2.6 模板的局限性（模板具体化）

- **核心：**模板通用性不是万能的 —— `a = b;` 遇到数组无法实现；`if (a > b)` 遇到自定义类型（如 `Person`）无法正常运行。解决方式是**提供具体化的模板（模板重载）**。
- **语法：**具体化以 `template<>` 开头，并指出具体类型。

```
template<class T>
bool myCompare(T& a, T& b) { return a == b; }

// 具体化：显示具体化的原型和定义以 template<> 开头，并通过名称指出类型
template<> bool myCompare(Person &p1, Person &p2) {
    return p1.m_Name == p2.m_Name && p1.m_Age == p2.m_Age;
}
```

- **规则：**具体化模板**优先于常规模板**被调用；利用具体化可解决自定义类型的通用化。
- **注意：**学习模板不是为了写模板，而是为了在 STL 中能够运用系统提供的模板。

3 类模板

● 3.1 类模板语法

- **核心：**建立一个通用类，类中的成员数据类型不具体制定，用一个虚拟类型代表。
- **语法：**

```
template<class NameType, class AgeType>
class Person {
public:
    Person(NameType name, AgeType age) { mName = name; mAge = age; }
    NameType mName;
    AgeType mAge;
};

Person<string, int> p("孙悟空", 999); // 必须显式指定类型
```

- **规则：**类模板与函数模板语法相似，在 `template` 声明后面加类，此类即为类模板。

● 3.2 类模板与函数模板区别

- **易错：**类模板没有自动类型推导的使用方式，`Person p("孙悟空", 1000);` 报错，必须 `Person<string, int> p("孙悟空", 1000);`。

- **规则：**类模板的模板参数列表可以有默认参数：`template<class NameType, class AgeType = int>`，于是可写 `Person<string> p("猪八戒", 999);`。

● 3.3 类模板中成员函数创建时机

- **核心：**普通类中的成员函数一开始就可以创建；**类模板中的成员函数在调用时才创建**（调用时才生成）。
- **易错/理解点：**类模板中写了 `void fun2(){ obj.showPerson2(); }`，若 `obj` 类型是 `Person1`，只要不调用 `m.fun2()` 就不会报错；一旦调用编译即出错。

```
template<class T>
class MyClass { public: T obj; void fun1(){ obj.showPerson1(); } void fun2(){ obj.showPerson2
(); } };

MyClass<Person1> m; m.fun1(); // 正确
// m.fun2(); // 编译出错：调用时才创建成员函数
```

● 3.4 类模板对象做函数参数

- **核心：**类模板实例化的对象向函数传参有三种方式 ——
 - **指定传入的类型：**直接显示对象的数据类型；
 - **参数模板化：**将对象中的类型参数变为模板进行传递；
 - **整个类模板化：**将这个对象类型整体模板化传递。

```
void printPerson1(Person<string,int>& p); // 1 指定传入类型
template<class T1,class T2> void printPerson2(Person<T1,T2>& p); // 2 参数模板化
template<class T> void printPerson3(T& p); // 3 整个类模板化
```

- **规则：**使用比较广泛的是**第一种：指定传入的类型**；参数模板化时可用 `typeid(T1).name()` 查看编译器推导出的类型。

● 3.5 类模板与继承

- **易错：**当子类继承的父类是一个类模板时，子类在声明时**必须指定出父类中 T 的类型**（如 `class Son : public Base<int>`），否则编译器无法给子类分配内存。
- **规则：**如果想灵活指定父类中 T 的类型，**子类也需变为类模板**：`template<class T1, class T2> class Son2 : public Base<T2>`。

● 3.6 类模板成员函数类外实现

- **核心：**类模板成员函数在类外实现时，**必须加上模板参数列表并写全作用域限定**。

```
template<class T1, class T2>
Person<T1, T2>::Person(T1 name, T2 age) { this->m_Name = name; this->m_Age = age; }

template<class T1, class T2>
void Person<T1, T2>::showPerson() { cout << m_Name << " " << m_Age << endl; }
```

● 3.7 类模板分文件编写

- **易错（问题根源）**：类模板中成员函数的创建时机在**调用阶段**，因此把声明写 .h、实现写 .cpp 时，主文件包含 .h 会导致**链接不到**成员函数实现。
- **解决方式**：
 - 直接 `#include "person.cpp"`（包含源文件）；
 - 把声明和实现写到同一个文件中，后缀名改为 `.hpp`（hpp 是约定名称，并非强制）。
- **规则**：主流解决方式是第二种（`.hpp`）。

● 3.8 类模板与友元

- **核心**：全局函数做友元的两种实现 ——
 - **类内实现**：直接在类内声明友元函数即可；
 - **类外实现**：需要**提前让编译器知道全局函数的存在**（前置声明类模板和函数模板），并且友元声明要写成模板实例形式。

```
template<class T1, class T2> class Person; // 前置声明类模板
template<class T1, class T2> void printPerson2(Person<T1,T2>& p) { /* ... */ }

template<class T1, class T2>
class Person {
    friend void printPerson(Person<T1,T2>& p) { /* 类内实现 */ }
    friend void printPerson2<>(Person<T1,T2>& p); // 类外实现，加 <>
};
```

- **建议**：推荐类内实现，用法简单且编译器可直接识别。

● 3.9 类模板案例 —— 通用数组类（MyArray）

- **需求**：可存储内置与自定义类型；数据存储在**堆区**；构造时传入容量；提供拷贝构造与 `operator=` 防止浅拷贝；提供尾插、尾删；支持下标访问；可获取当前元素个数与容量。
- **关键实现**：

```

MyArray(int capacity) { m_Capacity = capacity; m_Size = 0; pAddress = new T[capacity]; }
MyArray(const MyArray& arr) {                                     // 拷贝构造：深拷贝
    m_Capacity = arr.m_Capacity; m_Size = arr.m_Size;
    pAddress = new T[m_Capacity];
    for (int i = 0; i < m_Size; i++) pAddress[i] = arr.pAddress[i];
}
T& operator[](int index) { return pAddress[index]; }
void Push_back(const T& val) { if (m_Size == m_Capacity) return; pAddress[m_Size++] = val; }
void Pop_back() { if (m_Size == 0) return; m_Size--; }
~MyArray() { if (pAddress) { delete[] pAddress; pAddress = NULL; } }

```

- **易错：** `operator=` 中必须先判断并 `delete[]` 释放自身原有空间，再深拷贝并 `return *this`；若 T 是含指针的对象，其 `=` 也必须重载实现深拷贝，否则仍会浅拷贝。

1 STL 的诞生

- **核心**：为建立数据结构和算法的一套标准而诞生 STL；C++ 面向对象与泛型编程思想的目的都是提升**复用性**。

2 STL 基本概念

- **核心**：STL（Standard Template Library，标准模板库）从广义上分为 **容器(container)**、**算法(algorithm)**、**迭代器(iterator)** 三大块。
- **规则**：容器和算法之间通过**迭代器**进行无缝连接；STL 几乎所有的代码都采用了模板类或模板函数。

3 STL 六大组件

- **核心**：容器、算法、迭代器、仿函数、适配器（配接器）、空间配置器。
 - 容器：各种数据结构（vector、list、deque、set、map...），用来存放数据；
 - 算法：常用算法（sort、find、copy、for_each...）；
 - 迭代器：容器与算法之间的胶合剂；
 - 仿函数：行为类似函数，可作为算法的某种策略；
 - 适配器：用来修饰容器、仿函数或迭代器接口的东西；
 - 空间配置器：负责空间的配置与管理。

4 STL 中容器、算法、迭代器

- **容器分类**：
 - **序列式容器**：强调值的排序，每个元素均有固定的位置（vector、deque、list...）；
 - **关联式容器**：二叉树结构，各元素之间没有严格的物理顺序关系（set、map...）。
- **算法分类**：
 - **质变算法**：运算过程中会更改区间内元素的内容（拷贝、替换、删除...）；
 - **非质变算法**：运算过程中不会更改区间内元素内容（查找、计数、遍历、寻找极值...）。
- **迭代器**：提供一种方法依序寻访容器所含元素，而无需暴露容器内部表示；每个容器都有自己专属的迭代器；初学阶段可先理解为指针。
- **迭代器种类表**：

种类	功能	支持运算
输入迭代器	对数据的只读访问	只读，支持 <code>++</code> 、 <code>==</code> 、 <code>!=</code>
输出迭代器	对数据的只写访问	只写，支持 <code>++</code>
前向迭代器	读写，并能向前推进	读写，支持 <code>++</code> 、 <code>==</code> 、 <code>!=</code>
双向迭代器	读写，并能向前和向后操作	读写，支持 <code>++</code> 、 <code>--</code>
随机访问迭代器	可跳跃方式访问任意数据，功能最强	读写，支持 <code>++</code> 、 <code>--</code> 、 <code>[n]</code> 、 <code>-n</code> 、 <code><</code> 、 <code><=</code> 、 <code>></code> 、 <code>>=</code>

- **结论：** 常用容器中迭代器种类为 **双向迭代器**（list/set/map）和 **随机访问迭代器**（vector/deque/string）。

5 容器、算法、迭代器初识

5.1 vector 存放内置数据类型

- **核心：** 容器 `vector`、算法 `for_each`、迭代器 `vector<int>::iterator` 三件套。

```
vector<int> v;
v.push_back(10); v.push_back(20); v.push_back(30);
vector<int>::iterator pBegin = v.begin(), pEnd = v.end();
while (pBegin != pEnd) { cout << *pBegin << endl; pBegin++; } // 1 迭代器
for (vector<int>::iterator it = v.begin(); it != v.end(); it++) {} // 2 for 循环
for_each(v.begin(), v.end(), MyPrint); // 3 STL 算法
```

- **易错：** `v.begin()` 指向第一个元素，`v.end()` 指向**最后一个元素的下一个位置**（不是最后一个元素），循环条件必须是 `!= v.end()`。

5.2 vector 存放自定义数据类型

- **核心：** `vector<Person> v;` 存放对象；`vector<Person*> v;` 存放对象指针，遍历时用 `(*it).mName` / `it->mName`，指针容器用 `(*it)->mName`。

5.3 vector 容器嵌套容器

- **核心：** 容器中可以嵌套容器，遍历时用两层迭代器。

```
vector<vector<int>> v;  
for (vector<vector<int>>::iterator it = v.begin(); it != v.end(); it++)  
    for (vector<int>::iterator vit = (*it).begin(); vit != (*it).end(); vit++)  
        cout << *vit << " ";
```

STL —— string 容器

string Container

1 string 容器

1.1 string 基本概念

- **核心**: string 是 C++ 风格的字符串, **本质上是一个类**, 内部封装了 `char*`, 是一个 `char*` 型的容器。
- **与 `char*` 区别**: `char*` 是一个指针; string 是一个类, 内部管理 `char*` 所分配的内存, **不用担心复制越界和取值越界**。
- **特点**: 内部封装了很多成员方法, 如查找 find、拷贝 copy、删除 delete、替换 replace、插入 insert。

1.2 string 构造函数

```
string(); // 创建一个空字符串, 如 string str;  
string(const char* s); // 使用字符串 s 初始化  
string(const string& str); // 使用一个 string 对象初始化另一个  
string(int n, char c); // 使用 n 个字符 c 初始化
```

- **规则**: 多种构造方式没有可比性, 灵活使用即可。

1.3 string 赋值操作

```
string& operator=(const char* s); // char* 赋给当前字符串  
string& operator=(const string& s); // 字符串 s 赋给当前字符串  
string& operator=(char c); // 字符赋给当前字符串  
string& assign(const char* s);  
string& assign(const char* s, int n); // 把 s 的前 n 个字符赋给当前字符串  
string& assign(const string& s);  
string& assign(int n, char c); // 用 n 个字符 c 赋给当前字符串
```

- **结论**: string 的赋值方式很多, `operator=` 是最实用的。

● 1.4 string 字符串拼接

```
string& operator+=(const char* str);
string& operator+=(const char c);
string& operator+=(const string& str);
string& append(const char* s);
string& append(const char* s, int n);           // 拼接 s 的前 n 个字符
string& append(const string& s);
string& append(const string& s, int pos, int n); // s 中从 pos 开始 n 个字符
```

- 规则：重载版本很多，初学阶段记住几种即可。

● 1.5 string 查找和替换

```
int find(const string& str, int pos = 0) const; // 第一次出现位置，从 pos 开始
int find(const char* s, int pos = 0) const;
int find(const char* s, int pos, int n) const; // 查找 s 前 n 个字符第一次出现
int find(const char c, int pos = 0) const;
int rfind(const string& str, int pos = npos) const; // 最后一次出现位置
int rfind(const char* s, int pos = npos) const;
string& replace(int pos, int n, const string& str); // 替换 pos 起 n 个字符
```

- 易错：find 从左往右、rfind 从右往左；找到返回第一个字符的下标，找不到返回 -1（不是 0）。
- 注意：replace 替换时要指定「从哪个位置起、多少个字符、替换成什么字符串」。

● 1.6 string 字符串比较

```
int compare(const string& s) const;
int compare(const char* s) const;
```

- 规则：按字符的 ASCII 码比较；相等返回 0，大于返回 1，小于返回 -1。
- 注意：字符串比较主要用于判断两个字符串是否相等，判断谁大谁小意义不大。

● 1.7 string 字符存取

```
char& operator[](int n); // 通过 [] 方式取字符
char& at(int n);         // 通过 at 方法获取字符
```

- 规则：两种方式都可读可写（str[0] = 'x'; str.at(1) = 'x';）。

● 1.8 string 插入和删除

```
string& insert(int pos, const char* s);  
string& insert(int pos, const string& str);  
string& insert(int pos, int n, char c);    // 指定位置插入 n 个字符 c  
string& erase(int pos, int n = npos);     // 删除从 pos 开始的 n 个字符
```

- **规则：**插入和删除的起始下标都是从 0 开始。

● 1.9 string 子串

```
string substr(int pos = 0, int n = npos) const;    // 返回 pos 开始的 n 个字符
```

- **应用：**`email.substr(0, email.find("@"))` 取出邮箱用户名。
- **结论：**灵活使用求子串功能可在实际开发中获取有效信息。

string 常用接口汇总

类别	接口	说明
构造	<code>string()</code> / <code>string(const char*)</code> / <code>string(const string&)</code> / <code>string(int n, char c)</code>	空串、C 串、拷贝、n 个字符
赋值	<code>operator=</code> 、 <code>assign(s)</code> 、 <code>assign(s, n)</code> 、 <code>assign(n, c)</code>	最常用 <code>operator=</code>
拼接	<code>operator+=</code> 、 <code>append(s)</code> 、 <code>append(s, n)</code> 、 <code>append(s, pos, n)</code>	末尾追加
大小容量	<code>size()</code> 、 <code>length()</code> 、 <code>empty()</code> 、 <code>capacity()</code> 、 <code>resize(n)</code>	判空/长度
查找替换	<code>find</code> 、 <code>rfind</code> 、 <code>replace(pos, n, str)</code>	找不到返回 -1
比较	<code>compare(s)</code>	0 / 1 / -1
存取	<code>operator[]</code> 、 <code>at(n)</code>	可读可写
插入删除	<code>insert(pos, ...)</code> 、 <code>erase(pos, n)</code>	下标从 0 开始
子串	<code>substr(pos, n)</code>	截取子串

- **适用场景：**需要以类的方式安全、方便地管理字符串（查找、拼接、截取、替换）时，一律优先用 `string`，不要手写 `char*`。

STL —— vector 与 deque 容器

vector & deque

1 vector 容器

1.1 vector 基本概念

- **核心**：vector 数据结构与数组非常相似，也称为**单端数组**。
- **与普通数组区别**：数组是**静态空间**，而 vector 可以**动态扩展**。
- **动态扩展机制**：并不是在原空间之后续接新空间，而是**找更大的内存空间**，将原数据拷贝到新空间，再释放原空间。
- **规则**：vector 容器的迭代器是**支持随机访问**的迭代器。

1.2 vector 构造函数

```
vector<T> v; // 默认构造函数
vector(v.begin(), v.end()); // 将 v[begin(), end()) 区间元素拷贝给本身
vector(n, elem); // 将 n 个 elem 拷贝给本身
vector(const vector& vec); // 拷贝构造函数
```

1.3 vector 赋值操作

```
vector& operator=(const vector& vec); // 重载等号
assign(beg, end); // 将 [beg, end) 区间数据拷贝赋值给本身
assign(n, elem); // 将 n 个 elem 拷贝赋值给本身
```

1.4 vector 容量和大小

```
empty(); // 判断容器是否为空
capacity(); // 容器的容量
size(); // 返回容器中元素的个数
resize(int num); // 重新指定长度为 num，变长则用默认值填充，变短则删除超出元素
resize(int num, elem); // 变长时用 elem 填充新位置
```

- **易错**：`capacity()` 是容量、`size()` 是元素个数，二者通常不相等；`resize` 改变的是 `size`，`reserve` 改变的是 `capacity`。

● 1.5 vector 插入和删除

```
push_back(ele);           // 尾部插入元素
pop_back();               // 删除最后一个元素
insert(const_iterator pos, ele); // 迭代器指向位置 pos 插入元素 ele
insert(const_iterator pos, int count, ele); // pos 插入 count 个 ele
erase(const_iterator pos); // 删除迭代器指向的元素
erase(const_iterator start, const_iterator end); // 删除区间元素
clear();                  // 清空容器
```

● 1.6 vector 数据存取

```
at(int idx); // 返回索引 idx 所指的数据
operator[];  // 返回索引 idx 所指的数据
front();     // 返回容器中第一个数据元素
back();      // 返回容器中最后一个数据元素
```

- **结论：**除迭代器外，`[]` 和 `at` 也可获取元素；`front` 取首元素、`back` 取尾元素。

● 1.7 vector 互换容器

```
swap(vec); // 将 vec 与本身的元素互换
```

- **应用：**`vector<int>(v).swap(v);` 用匿名对象与自身交换，实现**收缩内存**（等价于 C++11 的 `shrink_to_fit()`）；`resize(3)` 后 size 变小但 capacity 不变。
- **结论：**swap 可以使两个容器互换，也可达到实用的收缩内存效果。

● 1.8 vector 预留空间

```
reserve(int len); // 容器预留 len 个元素长度，预留位置不初始化，元素不可访问
```

- **易错：**`reserve` 只预留 capacity，**不初始化、不可访问**这些位置；`resize` 才真正改变元素个数并构造元素。
- **结论：**数据量较大时，一开始用 reserve 预留空间可显著减少动态扩展（拷贝）次数。

vector 常用接口汇总

类别	接口	说明
构造	<code>vector<T> v;</code> <code>vector(beg,end);</code> <code>vector(n,elem);</code> <code>vector(const vector&)</code>	四种构造
赋值	<code>operator=</code> 、 <code>assign(beg,end)</code> 、 <code>assign(n,elem)</code>	——
容量大小	<code>empty()</code> 、 <code>capacity()</code> 、 <code>size()</code> 、 <code>resize(num)</code> 、 <code>resize(num,elem)</code> 、 <code>reserve(len)</code>	<code>capacity</code> ≠ <code>size</code>
插入删除	<code>push_back</code> 、 <code>pop_back</code> 、 <code>insert(pos,ele)</code> 、 <code>insert(pos,count,ele)</code> 、 <code>erase(pos)</code> 、 <code>erase(beg,end)</code> 、 <code>clear</code>	位置用 迭代器
数据存取	<code>at(idx)</code> 、 <code>operator[]</code> 、 <code>front()</code> 、 <code>back()</code>	——
互换	<code>swap(vec)</code>	可收缩内存
排序	<code>sort(v.begin(), v.end())</code> (全局算法, 需 <code><algorithm></code>)	随机访问迭代器才可用

- **适用场景**: 需要频繁随机访问、主要在尾部增删的连续存储场景, 首选 `vector`。

2 deque 容器

2.1 deque 容器基本概念

- **核心**: `deque` 是**双端数组**, 可以对头端进行插入删除操作。
- **与 `vector` 区别**:
 - `vector` 对**头部**插入删除效率低, 数据量越大效率越低;
 - `deque` 对**头部**插入删除速度比 `vector` 快;
 - `vector` 访问元素的速度比 `deque` 快 (与内部实现有关) 。
- **内部原理**: `deque` 内部有个**中控器**, 维护每段缓冲区的内容, 缓冲区中存放真实数据; 中控器维护每个缓冲区的地址, 使得使用 `deque` 时像一片连续的内存空间。
- **规则**: `deque` 容器的迭代器也是**支持随机访问**的。

2.2 deque 构造函数

```
deque<T> deqT;           // 默认构造形式
deque(beg, end);         // 将 [beg, end) 区间元素拷贝给本身
deque(n, elem);          // 将 n 个 elem 拷贝给本身
deque(const deque& deq); // 拷贝构造函数
```

● 2.3 deque 赋值操作

```
deque& operator=(const deque& deq);  
assign(beg, end);  
assign(n, elem);
```

● 2.4 deque 大小操作

```
deque.empty();           // 判断容器是否为空  
deque.size();            // 返回容器中元素的个数  
deque.resize(num);       // 重新指定长度，变长用默认值填充，变短删除超出元素  
deque.resize(num, elem); // 变长用 elem 填充
```

- 易错：deque 没有容量的概念（没有 `capacity()`，也没有 `reserve()`）。

● 2.5 deque 插入和删除

```
// 两端插入操作  
push_back(elem);  push_front(elem);  pop_back();  pop_front();  
  
// 指定位置操作  
insert(pos, elem);           // 在 pos 插入 elem，返回新数据的位置  
insert(pos, n, elem);        // 在 pos 插入 n 个 elem，无返回值  
insert(pos, beg, end);       // 在 pos 插入 [beg,end) 区间数据，无返回值  
clear();                   // 清空容器  
erase(beg, end);            // 删除 [beg,end) 区间数据，返回下一个数据位置  
erase(pos);                 // 删除 pos 位置数据，返回下一个数据位置
```

- 规则：插入和删除提供的位置是迭代器。

● 2.6 deque 数据存取

```
at(int idx);  operator[];  front();  back();
```

● 2.7 deque 排序

```
sort(d.begin(), d.end()); // 需 #include <algorithm>
```

- 规则：sort 算法非常实用，使用时包含头文件 `algorithm` 即可。

deque 常用接口汇总

类别	接口	说明
构造	<code>deque<T> d;</code> <code>deque(beg,end);</code> <code>deque(n,elem);</code> <code>deque(const deque&)</code>	与 <code>vector</code> 基本一致
赋值	<code>operator=</code> 、 <code>assign(beg,end)</code> 、 <code>assign(n,elem)</code>	——
大小	<code>empty()</code> 、 <code>size()</code> 、 <code>resize(num)</code> 、 <code>resize(num,elem)</code>	无 capacity/resize
插入删除	<code>push_back</code> 、 <code>push_front</code> 、 <code>pop_back</code> 、 <code>pop_front</code> 、 <code>insert(pos,...)</code> 、 <code>erase(pos)</code> 、 <code>erase(beg,end)</code> 、 <code>clear</code>	头尾双向操作
数据存取	<code>at(idx)</code> 、 <code>operator[]</code> 、 <code>front()</code> 、 <code>back()</code>	支持随机访问
排序	<code>sort(d.begin(), d.end())</code>	全局算法

- **适用场景：**需要频繁在头部和尾部同时插入删除时用 `deque`；只做随机访问和尾插则用 `vector`。

3 案例 —— 评委打分

- **描述：**5 名选手 ABCDE，10 个评委分别给每名选手打分，**去掉最高分、去掉最低分，取平均分。**
- **实现步骤：**
 - 创建五名选手放到 `vector` 中；
 - 遍历 `vector` 取出每个选手，用 `for` 循环把 10 个评分存入 `deque` 容器；
 - 用 `sort` 对 `deque` 中分数排序，用 `pop_back()`、`pop_front()` 去掉最高和最低分；
 - 遍历 `deque` 累加总分；
 - 求平均分并写回选手对象。
- **要点：**随机数需 `srand((unsigned int)time(NULL));`，分数为 `rand() % 41 + 60` (60~100)。
- **结论：**选取不同的容器操作数据，可以提升代码效率 (**`vector` 存对象、`deque` 头尾删除**)。

STL —— stack、queue 与 list 容器

stack, queue & list

1 stack 容器

1.1 stack 基本概念

- **核心**：stack 是一种**先进后出 (FILO)** 的数据结构，它只有一个出口。
- **易错**：栈中只有顶端的元素才可以被外界使用，因此**栈不允许有遍历行为**。
- **术语**：进入数据称为**入栈 push**，弹出数据称为**出栈 pop**。

1.2 stack 常用接口

```
// 构造函数
stack<T> stk;           // 默认构造
stack(const stack& stk); // 拷贝构造
// 赋值操作
stack& operator=(const stack& stk);
// 数据存取
push(elem); // 向栈顶添加元素
pop();       // 从栈顶移除第一个元素
top();       // 返回栈顶元素
// 大小操作
empty();     // 判断堆栈是否为空
size();      // 返回栈的大小
```

```
while (!s.empty()) { cout << s.top() << endl; s.pop(); } // 遍历栈的唯一方式
```

类别	接口	说明
构造/赋值	stack<T> stk; stack(const stack&) operator=	——
数据存取	push(elem)、pop()、top()	只有栈顶可访问
大小	empty()、size()	——
迭代器	无	不支持遍历、不支持 sort

- **适用场景：**需要**后进先出**的场景（逆序、括号匹配、递归模拟、表达式求值）。

2 queue 容器

2.1 queue 基本概念

- **核心：**queue 是一种**先进先出（FIFO）**的数据结构，它有**两个出口**。
- **规则：**队列允许从一端新增元素、从另一端移除元素；只有**队头和队尾**可以被外界使用。
- **易错：**队列**不允许有遍历行为，不提供迭代器**，更不支持随机访问。

2.2 queue 常用接口

```
// 构造函数
queue<T> que;           // 默认构造
queue(const queue& que); // 拷贝构造

// 赋值操作
queue& operator=(const queue& que);

// 数据存取
push(elem); // 往队尾添加元素
pop();      // 从队头移除第一个元素
back();     // 返回最后一个元素
front();    // 返回第一个元素

// 大小操作
empty();    // 判断队列是否为空
size();     // 返回队列大小
```

类别	接口	说明
构造/赋值	queue<T> que; queue(const queue&) operator=	——
数据存取	push(elem)、pop()、front()、back()	队头 front / 队尾 back
大小	empty()、size()	——
迭代器	无	不支持遍历

- **适用场景：**需要**先进先出**的场景（任务排队、BFS、消息缓冲）。

3 list 容器

3.1 list 基本概念

- **核心**：list 将数据进行**链式存储**；数据元素的逻辑顺序通过链表中的指针链接实现。
- **组成**：链表由一系列**结点**组成，结点 = 数据域（存数据元素）+ 指针域（存下一个结点地址）；STL 中的 list 是**双向循环链表**。
- **优点**：采用动态存储分配，不会造成内存浪费和溢出；插入和删除十分方便，修改指针即可，不需要移动大量元素。
- **缺点**：空间（指针域）和时间（遍历）额外耗费较大。
- **易错**：链表存储空间不连续，因此 **list 的迭代器只支持前移和后移，属于双向迭代器**，不支持随机访问。
- **重要性质**：list 的**插入和删除都不会造成原有 list 迭代器的失效**，这在 vector 中是不成立的。

3.2 list 构造函数

```
list<T> lst;           // 默认构造形式
list(beg, end);        // 将 [beg, end) 区间元素拷贝给本身
list(n, elem);         // 将 n 个 elem 拷贝给本身
list(const list& lst);  // 拷贝构造函数
```

3.3 list 赋值和交换

```
assign(beg, end);      // 将 [beg, end) 区间数据拷贝赋值给本身
assign(n, elem);       // 将 n 个 elem 拷贝赋值给本身
list& operator=(const list& lst);
swap(lst);             // 将 lst 与本身的元素互换
```

3.4 list 大小操作

```
size();               // 返回容器中元素的个数
empty();              // 判断容器是否为空
resize(num);          // 重新指定长度，变长用默认值填充，变短删除超出元素
resize(num, elem);    // 变长用 elem 填充
```

● 3.5 list 插入和删除

```
push_back(elem);    pop_back();        // 尾部增删
push_front(elem);   pop_front();       // 头部增删
insert(pos, elem);   // pos 位置插入 elem 的拷贝, 返回新数据的位置
insert(pos, n, elem); insert(pos, beg, end);
clear();             // 移除容器的所有数据
erase(beg, end);     // 删除区间数据, 返回下一个数据的位置
erase(pos);          // 删除 pos 位置数据, 返回下一个数据的位置
remove(elem);        // 删除容器中所有与 elem 值匹配的元素
```

- **注意:** `remove(elem)` 会删除**所有**匹配元素, 这是 list 独有的接口。

● 3.6 list 数据存取

```
front();    // 返回第一个元素
back();     // 返回最后一个元素
```

- **易错:** list 容器中不可以通过 `[]` 或 `at` 方式访问数据。
- **易错:** list 迭代器是双向迭代器, **不支持随机访问**, `it = it + 1;` 错误, 只能 `it++` / `it--`。

● 3.7 list 反转和排序

```
reverse();    // 反转链表 (成员函数)
sort();       // 链表排序 (成员函数), 默认从小到大
sort(myCompare); // 指定规则, 如从大到小
```

- **易错:** list 必须使用成员函数 `sort()`, 不能用全局 `std::sort` (全局 sort 要求随机访问迭代器)。

● 3.8 排序案例

- **描述:** 对 Person (姓名、年龄、身高) 排序, **年龄升序, 年龄相同时身高降序**。

```
bool ComparePerson(Person& p1, Person& p2) {
    if (p1.m_Age == p2.m_Age) return p1.m_Height > p2.m_Height;
    return p1.m_Age < p2.m_Age;
}

L.sort(ComparePerson);
```

- **易错:** 对于自定义数据类型, 必须指定排序规则, 否则编译器不知道如何排序。

- **结论：**高级排序只是在排序规则上再做一次逻辑规则制定，并不复杂。

list 常用接口汇总

类别	接口	说明
构造	<code>list<T> lst;</code> <code>list(beg,end);</code> <code>list(n,elem);</code> <code>list(const list&)</code>	——
赋值 交换	<code>operator=</code> 、 <code>assign(beg,end)</code> 、 <code>assign(n,elem)</code> 、 <code>swap(lst)</code>	——
大小	<code>size()</code> 、 <code>empty()</code> 、 <code>resize(num)</code> 、 <code>resize(num,elem)</code>	无 capacity
插入 删除	<code>push_back</code> 、 <code>pop_back</code> 、 <code>push_front</code> 、 <code>pop_front</code> 、 <code>insert(pos,...)</code> 、 <code>erase(pos)</code> 、 <code>erase(beg,end)</code> 、 <code>remove(elem)</code> 、 <code>clear</code>	头尾都能增删
数据 存取	<code>front()</code> 、 <code>back()</code>	无 <code>[]</code> 、无 <code>at</code>
反转 排序	<code>reverse()</code> 、 <code>sort()</code> 、 <code>sort(仿函数)</code>	成员函数，非全局算法

- **适用场景：**需要频繁在任意位置插入删除且不移动大量元素时用 list；代价是不能随机访问。

STL —— set 与 map 容器

set & map

1 set / multiset 容器

1.1 set 基本概念

- **核心**：所有元素都会在**插入时自动被排序**（默认从小到大）。
- **本质**：set/multiset 属于**关联式容器**，底层结构用**二叉树**实现。
- **区别**：set **不允许**容器中有重复元素；multiset **允许**有重复元素。

1.2 set 构造和赋值

```
set<T> st;                // 默认构造函数
set(const set& st);        // 拷贝构造函数
set& operator=(const set& st); // 重载等号操作符
```

- **注意**：set 容器插入数据用 `insert`；插入的数据会**自动排序**，不能用 `push_back`。

1.3 set 大小和交换

```
size();    // 返回容器中元素的数目
empty();   // 判断容器是否为空
swap(st);  // 交换两个集合容器
```

1.4 set 插入和删除

```
insert(elem);    // 在容器中插入元素
clear();         // 清除所有元素
erase(pos);      // 删除 pos 迭代器所指元素，返回下一个元素的迭代器
erase(beg, end); // 删除区间 [beg, end) 的所有元素，返回下一个元素的迭代器
erase(elem);     // 删除容器中值为 elem 的元素
```

● 1.5 set 查找和统计

```
find(key);    // 查找 key 是否存在, 存在返回该键元素的迭代器, 不存在返回 set.end()
count(key);   // 统计 key 的元素个数
```

- **规则:** 对于 set, count 结果为 **0 或 1**; find 返回的是**迭代器**。

● 1.6 set 和 multiset 区别

- **规则:** set 不可以插入重复数据、multiset 可以; **set 插入数据的同时会返回插入结果** (表示是否插入成功), multiset 不会检测数据。

```
pair<set<int>::iterator, bool> ret = s.insert(10);
if (ret.second) cout << "插入成功" << endl;    // 重复插入时 ret.second 为 false
```

- **结论:** 不允许插入重复数据用 set; 需要插入重复数据用 multiset。

● 1.7 pair 对组创建

```
pair<type, type> p(value1, value2);
pair<type, type> p = make_pair(value1, value2);
// 访问: p.first、p.second
```

● 1.8 set 容器排序

- **核心:** 改变 set 默认排序规则的主要技术点是**利用仿函数**。

```
class MyCompare { public: bool operator()(int v1, int v2) { return v1 > v2; } };
set<int, MyCompare> s2;                                     // 内置类型: 从大到小
set<Person, comparePerson> s;                             // 自定义类型: 必须指定规则
```

- **易错:** 对于**自定义数据类型**, set 必须指定排序规则才可以插入数据。
- **注意:** set 的元素值就是它的 key, 不能通过迭代器修改 (迭代器为 const 性质), 要改只能先删除再插入。

set / multiset 常用接口汇总

类别	接口	说明
构造赋值	<code>set<T> st;</code> <code>set(const set&)</code> <code>operator=</code>	——
大小交换	<code>size()</code> 、 <code>empty()</code> 、 <code>swap(st)</code>	——
插入	<code>insert(elem)</code>	返回 <code>pair<iterator,bool></code> (set)
删除	<code>erase(pos)</code> 、 <code>erase(beg,end)</code> 、 <code>erase(elem)</code> 、 <code>clear()</code>	——
查找统计	<code>find(key)</code> (返回迭代器)、 <code>count(key)</code> (0 或 1)	无 <code>operator[]</code>
排序规则	<code>set<T, 仿函数></code>	默认从小到大
数据存取	无 <code>[]</code> / <code>at</code> / <code>front</code> / <code>back</code>	靠迭代器

- **适用场景：**需要**自动去重** + **自动排序** + **快速查找**的集合数据（成绩排名、黑名单、去重统计）用 set；允许重复用 multiset。

2 map / multimap 容器

2.1 map 基本概念

- **核心：**map 中**所有元素都是 pair**；pair 中第一个元素为 **key（键值）**，起索引作用，第二个元素为 **value（实值）**。
- **规则：**所有元素都会根据元素的**键值自动排序**（默认按 key 从小到大）。
- **本质：**map/multimap 属于**关联式容器**，底层结构用二叉树实现。
- **优点：**可以根据 key 值快速找到 value 值。
- **区别：**map **不允许**容器中有重复 key 值元素；multimap **允许**有重复 key 值元素。

2.2 map 构造和赋值

```
map<T1, T2> mp; // map 默认构造函数
map(const map& mp); // 拷贝构造函数
map& operator=(const map& mp); // 重载等号操作符
```

- **注意：**map 中所有元素都是**成对出现**，插入数据时要使用**对组 (pair)**。

● 2.3 map 大小和交换

```
size();    // 返回容器中元素的数量  
empty();   // 判断容器是否为空  
swap(st);  // 交换两个集合容器
```

● 2.4 map 插入和删除

```
// 四种插入方式  
m.insert(pair<int, int>(1, 10));           // 1  
m.insert(make_pair(2, 20));               // 2  
m.insert(map<int, int>::value_type(3, 30)); // 3  
m[4] = 40;                                // 4 不建议用于插入  
  
// 删除  
m.erase(m.begin());                      // 按迭代器删除  
m.erase(3);                              // 按键值删除  
m.erase(m.begin(), m.end()); m.clear();
```

- **易错：**`m[key]` 访问一个不存在的 key 时，会**自动创建一个 value 为默认值 (0/空) 的元素**，因此它既是插入手段也是隐蔽的 bug 来源；multimap 也**不支持** `operator[]`。
- **结论：**map 插入方式很多，记住其一即可。

● 2.5 map 查找和统计

```
find(key);    // 存在返回该键元素的迭代器，不存在返回 map.end()  
count(key);   // 统计 key 的元素个数 (map 为 0 或 1; multimap 可 >1)
```

- **应用：**multimap 中 `find(key)` + `count(key)` 配合循环，可取出同一 key 的所有元素。

● 2.6 map 容器排序

```
class MyCompare { public: bool operator()(int v1, int v2) { return v1 > v2; } };  
map<int, int, MyCompare> m;    // 仿函数实现按 key 从大到小
```

- **规则：**利用仿函数可以指定 map 容器的排序规则。
- **易错：**对于自定义数据类型，**map 必须指定排序规则，同 set 容器。**
- **注意：**map 的排序永远是**按 key**，不能按 value 排序。

map / multimap 常用接口汇总

类别	接口	说明
构造 赋值	<code>map<T1,T2> mp;</code> <code>map(const map&)</code> <code>operator=</code>	——
大小 交换	<code>size()</code> 、 <code>empty()</code> 、 <code>swap(st)</code>	——
插入	<code>insert(pair<...>)</code> 、 <code>insert(make_pair(...))</code> 、 <code>insert(map<K,V>::value_type(...))</code> 、 <code>m[key]=value</code>	元素为 pair
删除	<code>erase(pos)</code> 、 <code>erase(beg,end)</code> 、 <code>erase(key)</code> 、 <code>clear()</code>	可按 key 删除
查找 统计	<code>find(key)</code> 、 <code>count(key)</code>	返回迭代器 / 个数
存取	<code>(*it).first</code> 、 <code>(*it).second</code> 、 <code>it->first</code> 、 <code>it->second</code> 、 <code>m[key]</code>	无 <code>at</code> / <code>front</code> / <code>back</code>
排序 规则	<code>map<K,V,仿函数></code>	按 key 排序

- **适用场景：**需要键值对映射、按 key 快速查找与自动排序时用 map（字典、统计词频、配置表）；key 可重复用 multimap（一对多分组）。

3 案例 —— 员工分组

- **描述：**公司招聘 10 个员工（ABCDEFGHIJ），员工信息为姓名+工资，部门分为策划、美术、研发；随机分配部门和工资，通过 **multimap** 插入 `key(部门编号) → value(员工)`，最后分部门显示员工信息。
- **实现步骤：**
 - 创建 10 名员工放到 `vector` 中（`#define CEHUA 0 / MEISHU 1 / YANFA 2`）；
 - 遍历 vector 取出每个员工，随机分组 `int deptId = rand() % 3;`
 - 用 `m.insert(make_pair(deptId, *it));` 放入 multimap;
 - 分部门显示: `pos = m.find(CEHUA); count = m.count(CEHUA);` 循环输出 count 个元素。

```
multimap<int, Worker>::iterator pos = m.find(CEHUA);
int count = m.count(CEHUA), index = 0;
for (; pos != m.end() && index < count; pos++, index++)
    cout << pos->second.m_Name << " " << pos->second.m_Salary << endl;
```

- **结论：**当数据以键值对形式存在时，可以考虑用 map 或 multimap。

函数对象与谓词

Function Objects

1 函数对象

1.1 函数对象概念

- **核心：**重载了函数调用操作符 `()` 的类，其对象常称为函数对象；函数对象使用重载的 `()` 时行为类似函数调用，所以也叫**仿函数**。
- **本质：**函数对象（仿函数）是一个类，不是一个函数。
- **谓词：**返回 `bool` 类型的仿函数称为谓词（详见 4.2）。

1.2 函数对象使用

- **三大特点：**
 - 使用时可以像普通函数那样调用，可以有参数，可以有返回值；
 - 函数对象超出普通函数的概念，可以有**自己的状态**（成员变量）；
 - 函数对象**可以作为参数传递**。

```
class MyPrint {  
public:  
    MyPrint() { count = 0; }  
    void operator()(string test) { cout << test << endl; count++; }  
    int count;          // 自己的状态  
};  
MyPrint myPrint; myPrint("hello world"); cout << myPrint.count; // 1
```

- **注意：**仿函数写法非常灵活，可以作为参数进行传递。

2 谓词

2.1 谓词概念

- **核心：**返回 `bool` 类型的仿函数称为谓词。
- **规则：**`operator()` 接受一个参数叫**一元谓词**；接受两个参数叫**二元谓词**。

● 2.2 一元谓词

```
struct GreaterFive { bool operator()(int val) { return val > 5; } };  
vector<int>::iterator it = find_if(v.begin(), v.end(), GreaterFive()); // 匿名函数对象
```

- **注意：**作为谓词传入算法时使用**匿名的函数对象**（`GreaterFive()`），不是类型名。

● 2.3 二元谓词

```
class MyCompare { public: bool operator()(int num1, int num2) { return num1 > num2; } };  
sort(v.begin(), v.end(), MyCompare()); // 改变排序策略，从大到小
```

- **结论：**参数只有两个的谓词称为二元谓词；谓词的作用是**改变算法的策略**。

3 内建函数对象

● 3.1 内建函数对象意义

- **核心：**STL 内建了一些函数对象，分为**算术仿函数**、**关系仿函数**、**逻辑仿函数**。
- **用法：**这些仿函数所产生的对象，用法和一般函数完全相同。
- **注意：**使用内建函数对象需要引入头文件 `#include <functional>`。

● 3.2 算术仿函数

```
template<class T> T plus<T>;           // 加法仿函数  
template<class T> T minus<T>;        // 减法仿函数  
template<class T> T multiplies<T>;    // 乘法仿函数  
template<class T> T divides<T>;       // 除法仿函数  
template<class T> T modulus<T>;       // 取模仿函数  
template<class T> T negate<T>;        // 取反仿函数
```

- **规则：**`negate` 是一元运算，其他都是二元运算。如 `plus<int> p; p(10, 20);`

● 3.3 关系仿函数

```
template<class T> bool equal_to<T>;           // 等于
template<class T> bool not_equal_to<T>;       // 不等于
template<class T> bool greater<T>;            // 大于
template<class T> bool greater_equal<T>;       // 大于等于
template<class T> bool less<T>;                // 小于
template<class T> bool less_equal<T>;          // 小于等于
sort(v.begin(), v.end(), greater<int>());      // 最常用
```

- **结论：**关系仿函数中最常用的就是 `greater<>`（大于），用于实现降序排序。

● 3.4 逻辑仿函数

```
template<class T> bool logical_and<T>;        // 逻辑与
template<class T> bool logical_or<T>;         // 逻辑或
template<class T> bool logical_not<T>;        // 逻辑非
vector<bool> v2; v2.resize(v.size());
transform(v.begin(), v.end(), v2.begin(), logical_not<bool>());
```

- **结论：**逻辑仿函数实际应用较少，了解即可。

STL 常用算法

STL Algorithms

- **概述**：算法主要由头文件 `<algorithm>`、`<functional>`、`<numeric>` 组成。
 - `<algorithm>`：所有 STL 头文件中最大的一个，涉及比较、交换、查找、遍历、复制、修改等；
 - `<numeric>`：体积很小，只包括几个在序列上进行简单数学运算的模板函数；
 - `<functional>`：定义了一些模板类，用以声明函数对象。

1 常用遍历算法

- **算法简介**：`for_each` 遍历容器；`transform` 搬运容器到另一个容器中。

• 1.1 for_each

```
for_each(iterator beg, iterator end, _func);
// beg 开始迭代器; end 结束迭代器; _func 函数或者函数对象

for_each(v.begin(), v.end(), print01);    // 普通函数
for_each(v.begin(), v.end(), print02());  // 函数对象
```

- **结论**：`for_each` 在实际开发中是最常用的遍历算法，需要熟练掌握。

• 1.2 transform

```
transform(iterator beg1, iterator end1, iterator beg2, _func);
// beg1/end1 源容器区间; beg2 目标容器开始迭代器; _func 函数或函数对象

vector<int> vTarget;
vTarget.resize(v.size());           // 必须提前开辟空间
transform(v.begin(), v.end(), vTarget.begin(), TransForm());
```

- **易错**：目标容器必须提前开辟空间（resize），否则无法正常搬运（会崩溃）。

2 常用查找算法

- **算法简介**：`find` 查找元素；`find_if` 按条件查找元素；`adjacent_find` 查找相邻重复元素；`binary_search` 二分查找法；`count` 统计元素个数；`count_if` 按条件统计元素个数。

● 2.1 find

```
find(iterator beg, iterator end, value);  
// 按值查找, 找到返回指定位置迭代器, 找不到返回结束迭代器 end()  
vector<int>::iterator it = find(v.begin(), v.end(), 5);  
if (it == v.end()) cout << "没有找到!" << endl;
```

- **易错:** find 查找自定义数据类型时, 该类必须重载 `operator==`, 否则编译失败。

● 2.2 find_if

```
find_if(iterator beg, iterator end, _Pred);  
// _Pred 函数或者谓词 (返回 bool 类型的仿函数)  
vector<Person>::iterator it = find_if(v.begin(), v.end(), Greater20());
```

- **结论:** find_if 按条件查找使查找更加灵活, 提供的仿函数可以改变不同的策略。

● 2.3 adjacent_find

```
adjacent_find(iterator beg, iterator end);  
// 查找相邻重复元素, 返回相邻元素的第一个位置的迭代器
```

- **结论:** 面试题中若出现查找相邻重复元素, 记得用 STL 中的 adjacent_find 算法。

● 2.4 binary_search

```
bool binary_search(iterator beg, iterator end, value);  
// 查到返回 true 否则 false  
bool ret = binary_search(v.begin(), v.end(), 2);
```

- **易错:** 在无序序列中不可用, 查找的容器中元素必须是有序序列。
- **结论:** 二分查找效率很高, 但前提是有序。

● 2.5 count

```
count(iterator beg, iterator end, value);  
int num = count(v.begin(), v.end(), 4);
```

- **易错:** 统计自定义数据类型时, 需要配合重载 `operator==`。

● 2.6 count_if

```
count_if(iterator beg, iterator end, _Pred);  
  
int num = count_if(v.begin(), v.end(), Greater4());
```

- **结论：**按值统计用 count，按条件统计用 count_if。

3 常用排序算法

- **算法简介：**sort 对容器内元素排序；random_shuffle 洗牌（指定范围内元素随机调整次序）；merge 容器元素合并并存储到另一容器；reverse 反转指定范围的元素。

● 3.1 sort

```
sort(iterator beg, iterator end, _Pred);  
  
sort(v.begin(), v.end()); // 默认从小到大  
sort(v.begin(), v.end(), greater<int>()); // 从大到小
```

- **易错：**自定义类型排序必须提供比较规则（谓词/仿函数），且容器迭代器必须支持**随机访问**（list 不能用全局 sort）。
- **结论：**sort 属于开发中最常用的算法之一，需熟练掌握。

● 3.2 random_shuffle

```
random_shuffle(iterator beg, iterator end);  
  
srand((unsigned int)time(NULL)); // 记得加随机数种子  
random_shuffle(v.begin(), v.end());
```

- **注意：**使用时记得加随机数种子，否则每次运行结果相同（新标准中该算法已被 std::shuffle 取代）。

● 3.3 merge

```
merge(iterator beg1, iterator end1, iterator beg2, iterator end2, iterator dest);  
  
vector<int> vtarget;  
vtarget.resize(v1.size() + v2.size()); // 目标容器需提前开辟空间  
merge(v1.begin(), v1.end(), v2.begin(), v2.end(), vtarget.begin());
```

- **易错：**两个容器必须是有序的，否则结果无意义；目标容器要提前 resize 为两容器大小之和。

● 3.4 reverse

```
reverse(iterator beg, iterator end);
```

- **结论：**reverse 反转区间内元素，面试题可能涉及。

4 常用拷贝和替换算法

- **算法简介：**`copy` 容器内指定范围元素拷贝到另一容器；`replace` 旧元素修改为新元素；`replace_if` 满足条件的元素替换为新元素；`swap` 互换两个容器的元素。

● 4.1 copy

```
copy(iterator beg, iterator end, iterator dest);  
  
vector<int> v2; v2.resize(v1.size()); // 目标容器提前开辟空间  
copy(v1.begin(), v1.end(), v2.begin());
```

- **易错：**拷贝时目标容器记得提前开辟空间（resize）。

● 4.2 replace

```
replace(iterator beg, iterator end, oldvalue, newvalue);  
replace(v.begin(), v.end(), 20, 2000); // 将 20 全部替换成 2000
```

● 4.3 replace_if

```
replace_if(iterator beg, iterator end, _pred, newvalue);  
replace_if(v.begin(), v.end(), ReplaceGreater30(), 3000);
```

- **结论：**replace_if 按条件替换，可以利用仿函数灵活筛选满足的条件。

● 4.4 swap

```
swap(container c1, container c2); // 互换两个容器的元素
```

- **易错：**交换容器时，注意交换的容器要同种类型。

5 常用算术生成算法

- **注意：**算术生成算法属于小型算法，使用时包含的头文件为 `#include <numeric>`。

- **算法简介:** `accumulate` 计算容器元素累计总和; `fill` 向容器中添加元素。

• 5.1 accumulate

```
accumulate(iterator beg, iterator end, value);    // value 为起始值  
  
int total = accumulate(v.begin(), v.end(), 0);
```

- **注意:** 头文件是 `numeric`, 这个算法很实用; 起始值类型决定结果类型 (要小数结果应传 `0.0`)。

• 5.2 fill

```
fill(iterator beg, iterator end, value);  
  
vector<int> v;  v.resize(10);  fill(v.begin(), v.end(), 100);
```

- **注意:** fill 要求容器已有元素 (先 resize), 否则无空间可填充。

6 常用集合算法

- **算法简介:** `set_intersection` 求交集; `set_union` 求并集; `set_difference` 求差集。

• 6.1 set_intersection

```
set_intersection(beg1, end1, beg2, end2, dest);  
  
vTarget.resize(min(v1.size(), v2.size()));    // 取较小值开辟空间  
  
vector<int>::iterator itEnd =  
    set_intersection(v1.begin(), v1.end(), v2.begin(), v2.end(), vTarget.begin());  
for_each(vTarget.begin(), itEnd, myPrint());
```

- **易错:** 求交集的两个集合**必须是有序序列**; 目标容器开辟空间需**取两个容器中的较小值**。
- **规则:** 返回值是**输出区间的结束位置**, 遍历时必须以它作为结束迭代器, 否则会打印出目标容器中的无效尾部元素。

• 6.2 set_union

```
set_union(beg1, end1, beg2, end2, dest);  
  
vTarget.resize(v1.size() + v2.size());    // 两个容器大小之和
```

- **规则:** 两个集合必须有序; 目标容器空间需**两个容器相加**; 返回值同样是并集输出区间的结束位置。

6.3 set_difference

```
set_difference(beg1, end1, beg2, end2, dest);
vTarget.resize(max(v1.size(), v2.size())); // 取较大值开辟空间
```

- **规则**：两个集合必须有序；目标容器空间需**取两个容器中的较大值**；返回值是差集输出区间的结束位置。
- **注意**：差集是**有方向**的—— `set_difference(v1,v2)` 与 `set_difference(v2,v1)` 结果不同。

7 附：容器选型速查（一句话结论）

容器	结构/迭代器	一句话适用场景
string	类封装 char*，随机访问	以面向对象方式安全管理字符串（查找/拼接/截取/替换）
vector	单端数组，随机访问	频繁随机访问、主要在尾部增删（首选默认容器）
deque	双端数组，随机访问，无 capacity	需要在 头尾两端 频繁插入删除
stack	适配器， 无迭代器	先进后出：逆序、括号匹配、递归模拟
queue	适配器， 无迭代器	先进先出：任务排队、BFS、消息缓冲
list	双向循环链表，双向迭代器	频繁在任意位置插入删除且不移动元素；不能随机访问
set / multiset	红黑树，双向迭代器，自动排序	自动去重+排序+快速查找（multiset 允许重复）
map / multimap	红黑树，按 key 排序，元素为 pair	键值对映射、按 key 快速查找（multimap 允许 key 重复）

8 高频易错点速记

1. 模板必须确定出 T 才能使用；自动类型推导要求类型一致。
2. 模板不能对自定义类型做通用比较/赋值，需要**具体化**（`template<>`）。
3. 类模板**没有自动类型推导**，但**可以有默认参数**。
4. 类模板成员函数**调用时才创建**（因此分文件编写会链接不到，需包含 .cpp 或改为 .hpp）。
5. 类模板继承必须指定父类 T 类型；类外实现成员函数必须加模板参数列表；友元类外实现需提前声明。
6. `vector` 的 `reserve`（只改容量、元素不可访问）与 `resize`（改元素个数、会构造/删除元素）不同。
7. `vector` vs `deque`：deque 头插头删快、vector 随机访问快、deque 无 capacity。
8. `list` 不支持随机访问（无 `[]`、无 `at`、迭代器不能 `+n`），排序必须用成员 `sort()`。
9. `set` 插入自动排序、不允许重复，插入返回 `pair<iterator,bool>`，count 为 0/1，自定义类型必须给排序规则。
10. `map` 中所有元素是 pair，按 **key** 排序，`m[key]` 会**隐式插入**默认值元素。

11. 仿函数是重载了 `operator()` 的类；返回 `bool` 的仿函数叫谓词（一元/二元）。
12. `sort` 对自定义类型必须提供比较规则；`list` 用成员 `sort`，`greater<int>()` 可实现降序。
13. `transform` / `copy` / `merge` / 集合算法的目标容器**必须提前 `resize` 开辟空间**。
14. `binary_search` 与所有集合算法都要求**有序序列**。
15. `accumulate` / `fill` 头文件是 `<numeric>`；内建仿函数头文件是 `<functional>`。

期末应试重点

Exam Essentials

1 本章说明

本章综合《C++期末总结超详细知识点》与两份思维导图整理。原资料面向校内期末闭卷考试，明确说明「学校考试一般比较简单，更多在于**概念和语法**的考察，包括一些细节知识点在选择、判断题中出现」「基本不涉及偏难怪知识点」，其覆盖面约为考试知识点的 **90%**。

技巧

技巧：本章按「考试频次」重排了顺序，先看前四节（程序结构、数据类型细节、运算符与流程、指针与引用），再看面向对象四节。判断题与选择题的失分几乎都集中在「细节」上。

2 程序构成与编译过程

- 一个 C++ 程序可以由**一个或多个程序单位**构成，每一个程序单位作为一个文件；编译系统**分别对各个文件进行编译**，因此一个文件就是一个**编译单元**。
- 用高级语言编写的程序称为**源程序**，C++ 源程序以 `.cpp` 为后缀。
- 从源码到可执行文件的路径：**源程序** `.cpp` → **目标程序** `.obj` → **二进制文件** `.exe`。
- 编写 C++ 程序一般经过的步骤：**编辑** → **编译** → **连接** → **调试**。

考点

考点：区分「编译单元」「源程序/目标程序/可执行文件」「编译与连接的区别」。

3 数据类型的高频细节

3.1 整型与浮点型

类型	字节	说明
<code>int</code> / <code>unsigned int</code>	4	默认整型
<code>float</code>	4	提供 6 ~ 7 位有效数字
<code>double</code>	8	提供 15 ~ 16 位有效数字
<code>char</code>	1	存放的是 ASCII 编码而非字符本身
<code>bool</code>	1	<code>true</code> 本质是 1, <code>false</code> 本质是 0

易错

易错：运算的两个数中只要有一个是 `float`，结果类型就是 `double`——C++ 在运算时把所有 `float` 都按 `double` 处理，算完再按需转换。

3.2 常量的三种进制与浮点写法

- 十进制：`1357`、`-432`、`0`
- 八进制：以数字 `0` 开头，每位只能是 `0~7`，如 `051`；`05188` 非法
- 十六进制：以 `0x` 或 `0X` 开头，每位是 `0~9`、`a~f`，如 `0x1A`；`0xfg00` 非法
- 浮点数的两种表示：十进制小数形式（`21.456`）、指数形式（`3.2e1`）；`e` 后必须跟整数，`3.2e1.5` 非法
- 浮点常量后缀：无后缀为 `double`；`f` / `F` 为 `float`；`l` / `L` 为 `long double`

● 3.3 字符与字符串

写法	是否合法	原因
'a'	合法	单引号内单个字符
'AB'	非法	单引号内只能有一个字符
"a"	合法	双引号表示字符串
'a"	非法	引号不配对
'\n'	合法	转义字符视为一个字符
'\17'	合法	八进制转义 \ooo
'\1F3'	非法	八进制转义只能含 0~7

- 字符串 "abc" 在内存中占 **4 个字节**（3 个可见字符 + 结尾 '\0'），'\0' 不是字符串内容，只是结束标志。
- 字符串 "abc\n" 包含 **4 个字符**（\n 是一个转义字符），但在内存中占 **5 个字节**。
- 转义字符 "\1017bc" 中 \101 是八进制转义（可能表示一个字符），计算长度时要按「转义序列整体算一个字符」。所以这类题必须先识别转义序列，再数可见字符。

考点

常考：字符串长度（strlen）与占用字节数（sizeof）的差别，是所有版本试卷的必考点。

● 3.4 标识符

- 只能由字母、数字、下划线组成，**必须以字母或下划线开头**
- **不能包含 -**（连字符）、不能是关键字、区分大小写

4 运算符与表达式

● 4.1 算术与赋值

- 两个整数相除结果仍是整数，直接舍去小数部分，不四舍五入
- 取模 % 只能用于整型，两个小数不能取模
- 除法与取模的除数都不能为 0
- 前置 ++ 先自增再参与表达式；后置 ++ 先参与表达式再自增

● 4.2 比较与逻辑

- 比较运算的结果：「真」用 1 表示，「假」用 0 表示

- `!` 的优先级在三者中最高；**关系运算符的优先级高于逻辑运算符**
- `&&` 与 `||` 具有**短路特性**：`a && b` 中 a 为假则不算 b；`a || b` 中 a 为真则不算 b
- 非 0 即为真，0 即为假；真可转成 1，假可转成 0

易错

易错：`a > 2 && b = 3` 会被解析为 `((a > 2) && b) = 3` 从而报错——关系运算符优先级高于赋值，写复合条件要加括号。

4.3 类型转换

- 赋值时若左右类型不一致，会把右侧表达式的值转换成**左侧变量的类型**
- `int i = 1.0 / 2;` 结果是 0 (`1.0/2` 得 0.5，赋给 int 时舍去小数)
- `int a = 2.3;` 中 a 仍为整型，值为 2
- 强制转换不改变原变量的类型：`double score = 9.5; (int)score;` 之后 `score` 依然是 `double`
- 取值范围大小关系：`long double > double > float > long > int > short > char`，**char 的取值范围最小**
- 由取值范围小的类型转换为大的类型是安全的（可能损失精度的是反方向）

4.4 运算符结合性与优先级

- `a < b < c` 的结合性是**自左向右**：先算 `a < b` 得到 0 或 1，再与 c 比较
- 赋值表达式的值就是被赋的值，且赋值运算符是**自右向左**结合
- 五个**不能重载**的运算符：`.`（成员访问）、`.*`（成员指针访问）、`::`（作用域）、`sizeof`（长度）、`?:`（条件）
- 运算符重载不改变运算符的**优先级、结合性与操作数个数**
- 单目运算符（`!`、负号 `-`、`&`、`*`、`++`、`--`、`sizeof()`）优先级高于算术、关系、逻辑运算符

易错

易错：`!m % i` 与 `!(m % i)` 完全不同；`%` 是双目运算符，`!` 是单目运算符，优先级不同结果不同。

5 流程结构

- `switch` 的表达式类型**只能是整型或字符型**，且不能判断区间
- `case` 后若无 `break`，程序会一直向下执行（穿透）
- `do...while` **至少执行一次**循环体，这是与 `while` 的关键区别
- `for` 的三个表达式用**分号**分隔
- `continue` 只结束本次循环，`break` 跳出整个循环
- `if` 语句中用作判断的表达式可以是**任意表达式**（不必是关系表达式）

- 三目运算符 `表达式1 ? 表达式2 : 表达式3`，且 C++ 中它返回的是**变量（左值）**

考点

考点：给出一段带 `while`、`a<b<c`、复合赋值的小程序，判断输出结果——务必**逐步手工演算**，不要凭感觉。

6 数组

- 下标从 0 开始；定义时 `{}` 内数据不足，剩余元素自动补 0
- 二维数组可以省略**行数**，但**不能省略列数**
- 全部元素赋初值时才可以不指定第一维长度
- 不能给数组整体赋初值或整体赋值：`int a[10] = {1*10};` 与 `a = b;` 都是错的
- 字符数组不能整体赋值，只能逐元素赋值：`char c[5]; c = {'C','h'};` 错误
- 数组名是**常量指针**：可以修改它指向的内容，但不能修改它本身；`a[i]` 等价于 `*(a+i)`
- 对数组名 `sizeof` 得到整个数组的字节数；作为函数参数时数组名**退化为指针**
- C 风格字符串常用函数：`strcat`（连接）、`strcpy`（复制）、`strcmp`（比较）、`strlen`（长度），都在 `<cstring>` 中

易错

易错：`strlen()` 的返回类型是**无符号数**，因此 `if (strlen(a) - strlen(b) > 0)` 这种写法恒为真；正确做法是先分别存入 `int` 变量再比较。

7 函数

要点	结论
函数定义五要素	返回值类型、函数名、参数列表、函数体语句、 <code>return</code> 表达式
形参与实参	定义处是形参，调用处是实参
函数嵌套	不能嵌套定义，但可以嵌套调用
声明与定义	声明可以多次，定义只能一次
默认参数	某位置有默认值后， 从左向右必须连续 都有默认值
默认参数与声明	声明给了默认值，实现处就不能再写默认值
占位参数	只有类型没有名字，调用时必须填补该位置
函数重载	同一作用域 + 同名 + 参数类型/个数/顺序不同； 返回值不能作为重载条件
内联函数	函数首行左端加 <code>inline</code> ；适合规模小且被频繁调用的简单函数
函数模板	只适用于函数体相同、参数 个数相同 而类型不同的情况

易错

易错：构造函数可以重载，**析构函数不能重载**（因为不能有参数）。重载时若与默认参数同时使用，极易产生二义性。

8 指针与引用

8.1 指针

- 普通变量存放**数据**，指针变量存放**地址**；`&` 取地址，`*` 解引用
- 所有指针类型在 32 位系统下都占 **4 个字节**（与所指类型无关）
- 空指针（`NULL`）与野指针指向的内存都**不可访问**
- 指针运算：`int n = 17, *p = &n, m;`
 - `m = (*p)++;` → 先取 17 赋给 m，再让 n 自增为 18，结果 `m = 17, n = 18`
 - `m = *p++;` → 先取 17 赋给 m，再让**指针**后移，结果 `m = 17, n = 17` 且 p 指向未知空间
- `a[i]` 等价于 `*(a+i)`；数组名就是数组的起始地址，`p+1` 指向下一个元素

● 8.2 const 修饰指针的三种形态

写法	名称	指向可否修改	指向的值可否修改
<code>const int * p</code>	指向常量的指针	可以	不可以
<code>int * const p</code>	常指针 (指针常量)	不可以	可以
<code>const int * const p</code>	指向常量的常指针	不可以	不可以

- 判断技巧：按自右向左看 `const` 修饰的是谁。`const char* a` 中 `*a` 是常量、`a` 不是；`char* const a` 中 `a` 是常量、`*a` 不是。
- `int a[10]` 中数组名 `a` 就是常量指针——可以改 `a[i]`，不能改 `a` 本身。

考点

考点：`const char* ptr = "hello";` 说明 `ptr` 是指向字符串常量的指针——不能通过 `ptr` 修改内容，但可以改指向。这是判断题的高频原句。

● 8.3 引用

- 引用必须初始化，`int &c;` 错误
- 引用初始化后不能改绑到其他变量
- 引用做参数可以修饰实参；效果等同于地址传递但语法更简洁
- 引用做返回值时，函数调用可以作为左值；但不要返回局部变量的引用（局部变量在栈区，函数结束即释放）；返回引用时常用 `static` 变量

易错

易错：常成员函数的标准写法是 `void print() const;`——`const` 写在参数表之后、函数体之前。

9 面向对象高频细节

● 9.1 类的基础

- `struct` 与 `class` 唯一区别是默认访问权限：`struct` 默认 `public`，`class` 默认 `private`
- 类继承中默认的继承方式是私有继承（`class Son : Base` 等价于 `class Son : private Base`）
- 赋值运算符 `operator=` 不能被派生类继承
- 每个类有且仅有一个拷贝构造函数
- 作用域运算符 `::` 的功能是指出作用域的范围，还可以用来访问静态数据成员、指明成员属于哪个类
- 静态成员函数没有 `this` 指针；非静态成员函数才有
- 静态成员函数可以直接访问类的静态数据和静态函数成员，但访问非静态成员必须通过对象名

- 静态成员变量「类内定义（声明），类外初始化」

易错

易错：`sizeof(结构体)` 是所有成员占用字节数之和（按最大成员对齐时需考虑补位）；结构体定义只是声明了一种数据类型，不分配内存空间。

● 9.2 对象空间与 const

- 对象空间 = 对象的基本空间（非静态成员之和）+ 资源空间（指针指向的空间）
- 常量数据成员只能通过构造函数的初始化列表完成初始化：
`Circle::Circle(double r) : radius(r), PI(3.1415) {}`
- 常量成员函数只能读取数据成员，不能修改，也不能调用本类中没有用 `const` 修饰的成员函数
- 类外实现常量成员函数也要带 `const`
- `const` 可以用于区分重载函数
- 常对象的所有数据成员都不能修改；常对象只能调用它的常量成员函数
- 发生 `const` 重载时：普通对象调用非 `const` 版本，常对象调用 `const` 版本

● 9.3 构造与析构

- 构造函数：无返回值也不写 `void`、名字与类名相同、可以重载、创建对象时自动调用且只调一次
- 析构函数：无返回值也不写 `void`、名前加 `~`、不可以有参数因而不能重载、对象销毁前自动调用且只调一次
- 组合类（has-a）的构造顺序：①先调用成员类对象参数类型的拷贝构造创建形参（若形参为引用则不调用）②再调用组合成员的构造，顺序依照成员在类中声明的顺序③最后进入自己的函数体
- 析构顺序与构造顺序严格相反
- 派生类构造函数执行顺序：①基类构造函数（按被继承时声明的顺序，自左向右）②组合成员构造函数（按派生类中声明顺序）③派生类构造函数体
- 派生类不继承基类的构造函数与析构函数

考点

考点：给出继承 + 组合的类层次，要求写出完整构造/析构输出顺序——按「基类 → 成员 → 自身，析构反向」推。

● 9.4 继承与虚继承

- 公有继承的赋值兼容规则（实现运行时多态的基础）：派生类对象可以看作基类对象、可以为基类对象赋值、可以初始化基类对象的引用、派生类对象的指针可以指向基类对象（反之需要强制转换）
- 派生类声明与基类同名的新成员会产生同名隐藏；解决二义性可用「类名限定」`c1.A::f()` 或让同名成员隐藏

- 菱形继承：在菱形的腰部两个基类加 `virtual` 关键字即可形成虚基类
- 有虚基类的构造函数：不仅要调用直接基类的构造函数，也要调用最远基类（虚基类）的构造函数；若未列出，则调用该虚基类的默认构造函数

易错

易错：一条继承链上只有最远派生类负责构造虚基类，中间类对虚基类的构造调用会被忽略。

● 9.5 多态

- 编译时多态（静态绑定）：函数重载、运算符重载
- 运行时多态（动态绑定）：虚函数；前提是赋值兼容规则
- 虚函数必须是非静态成员函数
- 虚函数在类外实现时不需要再加 `virtual`（与 `static`、`friend` 规则一致）
- 虚函数的限制：
 - 静态成员函数不能声明为虚函数（它不属于某个具体对象）
 - 内联成员函数不能被声明为虚函数（内联函数代码明确，没有多态特征）
 - 构造函数不能是虚函数（虚函数需要在对象实例化之后调用，此时对象还未建立）
 - 析构函数应当声明为虚函数（易漏考点）：在具备继承—派生关系的多个类中，若通过基类指针释放派生类对象，析构函数必须为虚函数
- 纯虚函数：`virtual void show() = 0;` 仅为多态机制提供界面而无实体定义
- 抽象类：至少包含一个纯虚函数的类；抽象类不能生成对象

考点

考点：多态的三个条件——①有继承关系 ②子类重写父类的虚函数（返回值、函数名、参数列表完全一致）③父类指针或引用指向子类对象。

● 9.6 运算符重载的若干规定

- 不能改变运算符的操作数个数、优先级、结合性
- 重载运算符的函数不能有默认参数（否则改变了参数个数）
- 参数中至少有一个是类对象（或类对象的引用），不能全是标准类型
- 一般将单目运算符重载为成员函数，双目运算符重载为友元函数
 - 成员函数形式：`operator` 函数有 1 个参数
 - 友元函数形式：`operator` 函数有 2 个参数
- 只能把 `<<`（插入）和 `>>`（提取）重载为友元函数或普通函数，不能定义为成员函数（因为参数为两个）
- 类型转换运算符只能作为成员函数重载

- 单目运算符: `!`、`-` (负号)、`&`、`*`、`++`、`--`
- 双目运算符: `*`、`/`、`%`、`+`、`-`、`==`、`!=`、`<`、`>`、`<=`、`>=`、`&&`、`||`

考点

考点: `s = x + y` 被编译器解释为 `x.operator+(y)` (成员函数) 或 `operator+(x, y)` (友元/全局函数); 单目 `++` / `--` 与赋值 `=` 的重载都涉及 `*this` 的返回。

10 文件与输入输出流

- `<<` 是插入运算符, `>>` 是提取运算符
- `ofstream` 是 `ostream` 的派生类, **不是** `iostream` 的派生类
- `cerr` 是标准错误输出流, **没有缓冲**, 发送给它的内容立即被输出
- 文件流三种类: `ofstream` (写)、`ifstream` (读)、`fstream` (读写)
- 文件打开方式: `ios::in` (读)、`ios::out` (写, 文件不存在则建立, 存在则清空)、`ios::app` (追加)、`ios::binary` (二进制); 可用 `|` 组合
- 打开文件的两种方法: 用成员函数 `open()`, 或在定义文件流对象时指定打开方式
- `fail()` 判断文件是否打开失败; `eof()` 判断文件是否结束
- 标准输入流常用成员函数: `get()` 输入单个字符、`getline(字符指针, 字符个数 n)` 输入一行 (实际只存 `n-1` 个字符)、`eof()` 判断是否结束

易错

易错: `cin.getline(buf, n)` 最多读入 `n-1` 个字符, 要留一个位置给结束符。

11 考前冲刺建议

1. **概念与语法细节优先**: 本章中带「注意」「易错」「考点」标记的条目, 是选择题与判断题的直接来源。
2. **手写关键代码**: 构造函数/析构函数、深浅拷贝、运算符重载、多态与虚析构、模板与 STL 基本用法, 各写一遍比读十遍有效。
3. **演算程序阅读题**: 遇到 `while`、`a<b<c`、复合赋值、前置后置自增混合的题目, 一律在纸上逐步推演变量值。
4. **多做往年题**: 考试偏应试, 知识点掌握后务必通过做往年习题来查漏; 错题回填到本书第 25 章的易错清单中。

C/C++ 易错点清单

Common Pitfalls

1 本章说明

本章依据《C_++易错总结》整理，逐条列出选择题、判断题与程序阅读题中最容易出错的结论。**每一条都是可以直接被考察的陈述句**，建议考前逐条自测：先看题目、说出结论，再看解析。

技巧

技巧：本章按「作用域与指针 → 类型与常量 → 类与对象 → 运算符与优先级 → 存储与内存 → 易错题」排列，共 40 余条。全部能复述一遍，细节题基本不会失分。

2 作用域运算符 ::

- `::` 的功能是**指出作用域的范围**。
- 它可以用来**访问静态数据成员**、标明成员属于哪个类。
- `X::member` 表示 `member` 是属于 `X` 这个类的成员，而不是某个对象的成员。

3 指针与常量：一字之差的两种类型

`const char* ptr = "hello";` 中 `ptr` 的类型是 **指向字符串常量的指针**。

「指向字符串的常量指针」与「指向字符串常量的指针」的区别在于：**指针本身的值是否能修改**，判断方法是**按自右向左看** `const` 修饰的是谁。

写法	名称	含义	可否改指向	可否改内容
<code>const char* a</code>	指向字符串常量的指针	<code>const</code> 修饰 <code>char</code> ， <code>*a</code> 是常量	可以	不可以
<code>char* const a</code>	指向字符串的常量指针	<code>const</code> 修饰 <code>a</code> ， <code>a</code> 本身是常量	不可以	可以

- `const char* a` 等价于 `char const *a`；`*a` 是常量，但 `a` 不是。
- `char* const a` 中 `a` 是常量，但 `*a` 不是。
- `int a[10]` 中数组名 `a` 是一个**常量指针**：可以修改它指向的内容（`a[i] = 1`），但不能修改它本身（`a = ...` 错误）。

易错

易错: `const` 的位置决定语义。看到 `const type *p` 记「值不可改」, 看到 `type * const p` 记「指向不可改」。

4 常量成员函数的标准写法

已知 `print` 函数是一个类的常成员函数, 它无返回值, 正确表示为:

```
void print() const;
```

考点

考点: `const` 写在参数表之后、函数体之前; 写成 `const void print();` 是把 `const` 用在了返回类型上, 属于另一种完全不同的含义。

5 不能重载的五个运算符

C++ 中不能被重载的运算符有 5 个:

运算符	名称
<code>.</code>	成员访问运算符
<code>.*</code>	成员指针访问运算符
<code>::</code>	域运算符 (作用域运算符)
<code>sizeof</code>	长度运算符
<code>?:</code>	条件运算符

考点

考点: 运算符重载**不能改变**运算符的操作数个数、优先级与结合性; 重载函数的参数中至少有一个是类对象。

6 字符串长度与转义字符

字符串	字符数 (不含结束符)	占用字节数	说明
"abc"	3	4	结尾自动补 '\0'
"abc\n"	4	5	\n 是一个转义字符, 占 1 字节
"\\"	1	2	\\ 是一个反斜线字符

- 字符串 "0" 与字符 '0' 不同; 字符串不需要手写结束符, 编译器自动添加 '\0'。
- 八进制转义 \ooo 最多取 3 位、每位范围 0~7; 十六进制转义 \xhh 每位是 0~9、a~f。
- '\17' 合法 (八进制转义); '\1F3' 非法 (出现字母 F); 'AB' 非法 (单引号内只能一个字符)。

易错

易错: 数长度前一定先把转义序列圈出来。转义序列无论看起来几个符号, 都只算 1 个字符; 这也是「"abc\n" 是 4 个字符却占 5 个字节」的原因。

7 负数的原码、反码与补码

计算机中整数以补码形式存放。三种表示法 (以 8 位为例):

表示法	正数	负数规则
原码	符号位 0, 其余位为值, 如 00000001 = 1	符号位 1, 其余位为值的绝对值, 如 10000001 = -1
反码	与原码相同	符号位不变, 其余各位取反
补码	与原码相同	反码加 1

- 8 位原码范围: 正数 00000001 ~ 01111111 (0~127)、负数 11111111 ~ 10000001 (-127~-1)。
- 补码的意义: 把减法统一成加法, 且 0 只有唯一表示。

格式化输出:

说明符	含义
%d	十进制有符号整数
%o	按无符号整数输出八进制形式
%x / %X	按无符号整数输出十六进制形式 (小写 / 大写)

- 执行 printf("%o", -1); 输出的是一串 7 (32 位 int 时 -1 的补码为全 1, 按无符号八进制解释即为 3777777777), 而不是 -1 或 -177777。

易错

易错：负数按 `%o`、`%x` 输出时会先被当作**无符号数**解释，因此结果是一长串最大值，这是判断题与输出题的常见考点。

8 数据类型转换

已知：

```
char a = 'a';
float b = 2.3;
double c = 4.1;
int d;
d = a + b + c;
```

执行后 **d 的数据类型是 int**（`d` 本身的类型不会因赋值而改变，只是运算结果被转换成 `int`）。

关于类型转换的规则：

- 不同类型数值进行算术运算时，结果的类型取决于**操作数中优先级最高的类型**（取值范围最大的类型）。
- 字符 `'a'` 参与运算时会被自动转换为整数 **97**。
- `int a = 2.3;` 中 `a` 仍为整型，值为 2。
- **赋值时**，右侧表达式的值会被转换为**左侧变量的类型**。
- `int i = 1.0 / 2;` 结果为 0（先算 `1.0/2 = 0.5`，赋给 `int` 时舍去小数）。
- 强制类型转换**不改变原变量的类型**：`double score = 9.5; (int)score;` 之后 `score` 依旧是 `double`。
- 类型取值范围：`long double > double > float > long > int > short > char`，其中 **char 的取值范围最小**；把取值范围小的类型转换为大的类型是安全的。

易错

易错：「赋值后变量类型会变」是错误认识——变量的类型在定义时就固定了。

9 模板不是面向对象所特有的特性

下列不属于面向对象程序设计所特有的是 **模板**（泛型编程）。

考点

考点：面向对象三大特性是**封装、继承、多态**；模板/泛型编程是**另一套**编程范式，与面向对象并列。

10 二维数组的存储顺序

```
int a[3][3] = { {1,2}, {5}, {2,4,8} };
```

初始化后数组内容是：

行	内容
a[0]	1, 2, 0
a[1]	5, 0, 0
a[2]	2, 4, 8

因此「初始值 5 是数组 a 存储在内存中的**第 4 个元素**」——二维数组在内存中**按行优先连续存放**，不足的部分自动补 0。

易错

易错：{ {1,2}, {5}, {2,4,8} } 中每一行不足 3 个元素，其余位置补 0，但补的 0 **仍然占位置**，所以 5 的下标是 `a[1][0]`，在内存中排第 4 位。

11 语法细节速记

- `if` 语句中用做判断的表达式可以是**任意表达式**（不限定为关系表达式）。
- 八进制整型常量以 **0** 作为前缀。
- 每个类有**且仅有一个**拷贝构造函数。
- 计算机中内存器的最小存储单位是**比特 (bit)**，最基本的存储单位是**字节 (Byte)**，1 字节 = 8 比特。
- 关于 C/C++ 中的**非法常量**：

写法	是否合法	原因
05188	非法	八进制每位只能到 7
0x18	合法	十六进制以 0x 开头，每位 0~f
0xfg00	非法	含 f 之外的字母 g
'a'、'\\'、'\n'	合法	单引号内单个字符或合法转义
'a"、'ab'	非法	引号不配对 / 单引号内多个字符
3.2e1.5	非法	e 后面必须跟整数
10.0f	合法	f 后缀表示 float
10.0L	合法	L 后缀表示 long double
10.0	合法	无后缀为 double

易错

易错：不带 f / F、l / L 后缀的浮点常量类型为 **double**；f / F 为 float，l / L 为 long double。

- 标识符由字母、数字、下划线组成，必须以**字母或下划线**开头，不包括 -（连字符）。
- 特殊字符就是**转义字符**：\ooo 为八进制值对应的字符（\17 合法、\1F3 非法）；\xhh 为十六进制值对应的字符。

12 输入输出与文件

- << 是**插入运算符**，>> 是**提取运算符**。
- ofstream 不是 istream 的派生类。
- cerr 是标准**错误**输出流，**没有缓冲**，发送给它的内容立即被输出（对比 clog 有缓冲）。
- 关于流成员函数的功能：read 的功能是从输入流中读取指定**若干字节**到缓冲区（不是"若干行"）。
- 文件流三类：ofstream 写、ifstream 读、fstream 读写。

13 继承相关

- 类继承中，默认的继承方式是**私有继承**（class Son : Base 等价于 class Son : private Base）。
- C++ 中，**赋值操作函数（operator=）**不能被派生类继承。
- 公有继承下基类的私有成员**不可访问**，保护成员在派生类中仍是保护成员。

易错

易错：默认继承方式与被继承成员的默认访问权限是两回事——`struct` 默认 `public` 继承，`class` 默认 `private` 继承。

14 运算符重载的规则

- 运算符重载时，**优先级和操作数个数不变**。
- 重载不能改变运算符的**结合性**。
- 重载运算符的函数不能有**默认参数**（否则会改变参数个数，与上一条矛盾）。
- 参数不能全部是标准类型——**至少有一个是类对象或其引用**。
- 一般把**单目运算符重载为成员函数**、**双目运算符重载为友元函数**：
 - 成员函数形式 → `operator` 函数有 **1 个** 参数
 - 友元函数形式 → `operator` 函数有 **2 个** 参数
- `<<` 与 `>>` 只能重载为**友元函数或普通函数**，不能定义为成员函数。
- **类型转换运算符只能作为成员函数重载**。

15 类模板与模板类

类模板和模板类是不同的概念。

- **类模板**（`class template`）：是一个模板，是产生类的模具，如 `template<class T> class Person { ... };`，它本身不是类。
- **模板类**（`template class`）：是类模板**实例化后得到的具体的类**，如 `Person<string, int>`。

考点

考点：这是概念判断题的常见原句——「类模板」是模板，「模板类」是类，二者不可混用。

16 静态成员与 this 指针

- **静态成员函数可以直接访问类的静态数据和静态函数成员**，但访问**非静态成员**，必须通过对象名。
- **静态成员函数没有 this 指针**（它不属于任何对象）；**非静态成员函数才有 this 指针**。
- C++ 编译器给每个**非静态的成员函数**增加了一个隐藏的指针参数，让该指针指向当前对象（即运行时调用该函数的对象）；函数体内所有对成员变量的操作，都是通过该指针去访问。
- 非静态的成员函数**不能操作静态的成员**？——相反：**静态成员函数不能访问非静态成员**；非静态成员函数可以访问静态成员。做题时注意「谁能访问谁」的方向。

易错

易错：静态成员函数没有 `this`，因此无法知道“是哪个对象在调用”，这就是它不能访问非静态成员的根本原因。

17 存储区域与变量生存期

全局变量、静态全局变量、静态局部变量都存放在内存的静态存储区域。

- 静态局部变量（具有**记忆功能**）在下次进入函数前，值仍然保留。
- 自动变量（`auto`）在进入语句块时自动申请内存、退出时自动释放，**不具有记忆功能**，定义时**不会自动初始化**。
- 静态变量与程序**共存亡**；自动变量与程序块**共存亡**。

17.1 C 语言中变量的存储类型

存储类型	特点
<code>auto</code>	自动类型；未指定存储类型时隐含为 <code>auto</code> ；动态局部变量，形参也属于动态局部变量；不自动初始化
<code>register</code>	寄存器类型；现代编译器能自动把普通变量优化为寄存器变量；使程序运行更快；与自动变量一样属于动态局部变量
<code>static</code>	静态类型；与程序共存亡；静态全局变量只限本文件使用；静态局部变量具有记忆功能；在静态存储区分配内存
<code>extern</code>	外部类型；没有显式初始化的外部变量 自动初始化为 0

考点

考点：存储类型决定变量的**生存期**；`auto` / `register` 是动态局部，`static` / `extern` 与程序或文件同级。

18 结构体、共用体与枚举

18.1 结构体

- 结构体的定义**只是声明了一种数据类型**，并未声明结构体类型的变量，因而**不分配内存空间**。
- 结构体类型变量的内存容量是**所有成员内存容量之和**；存在对齐补位时，实际占用取「最大成员字节数 × 成员个数」的整数倍关系。
- 结构体变量的三种定义方式：
 - 先声明结构体模板，再定义变量：`struct Student { ... }; struct Student stu1;`

- 声明模板的同时定义变量: `struct Student { ... } stu1;`
- 用 `typedef` 为已有结构体类型定义新名字:
`typedef struct Student { ... } STUDENT; STUDENT stu1;`

• 结构体可以做**构造链表**、**结构体数组**、**结构体指针**的成员。

● 18.2 共用体 union

- 占用内存等于**占空间最多的那个成员变量**的大小。
- 每一瞬时只存放**一种类型**的成员（各成员共享同一块内存）。

● 18.3 枚举 enum

- 枚举描述的是一组**整型值的集合**。
- 一般第一个枚举常量的值为 **0**，第二个为 1，第 n 个为 n-1。
- `enum response { no, yes, none };` 中 `no` 为 0、`yes` 为 1、`none` 为 2。
- 定义一个枚举类型变量: `enum response answer = no;`
- **枚举类型的值都是整型而非字符串**。

易错

易错: 枚举成员是**整型常量**，不能直接输出成 `"yes"` 这样的字符串。

19 指针运算与传参

- `a[i]` 等价于 `*(a+i)`；间接寻址符 `*` 用于取出指针所指地址的内容。
- 指针运算示例:

```
int n = 17, *p = &n, m;
```

```
m = (*p)++; // 先取 n 的值 17 赋给 m，再让 n 自增: m=17, n=18
```

```
m = *p++; // 先取 17 赋给 m，再让 p 后移: m=17, n=17, p 指向未知空间
```

- 调用函数时，实参对应形参之间的数据传递方式是**单向值传递**。

易错

易错: `(*p)++` 改的是**所指变量的值**；`*p++` 改的是**指针本身**（因为 `++` 优先级高于 `*`，等价于 `*(p++)`）。

20 运算符结合性与优先级

- `a < b < c` 的结合性是**自左向右**：先算 `a < b` 得 0 或 1，再与 c 比较。
- 赋值运算符是**自右向左**结合；赋值表达式的值就是被赋予的值。
- `/` 运算要求参与运算的数据必须是**整型**时才能得到整除结果（说明整除的语义）。
- 运算符优先级从高到低（简化记忆）：

单目运算符（`!` `-` `++` `--` `sizeof`）

↓

算术运算符（`*` `/` `%` `→` `+` `-`）

↓

关系运算符（`>` `>=` `<` `<=` `==` `!=`）

↓

逻辑运算符（`&&` `||`，`!` 除外——`!` 属于单目最高级）

↓

赋值运算符（`=` `+=` `-=` `...`）

↓

逗号运算符

- `!m % i` 与 `!(m % i)` 不同（`!` 是单目、`%` 是双目，优先级 `!` 更高）。
- `a > 2 && b = 3` 会被解析成 `((a > 2) && b) = 3`，**会报错**——写复合条件要加括号。
- 关系运算符的优先级高于逻辑运算符，所以 `a >= 2 && a <= 9` 不必加括号也正确。

20.1 逻辑运算的短路特性

- `&&`：如果第一个运算量为假，则**不会**去计算第二个表达式。
- `||`：如果第一个运算量为真，则**不会**去计算第二个表达式。
- 非 0 的数表示为真，0 表示为假；真可以转换为数字 1，假可以转换为数字 0。

21 字符串函数与字符输入

- 字符串用 `gets()` 可以接受**带空格**的字符串。
- `strlen()` **不能直接比较字符串的大小**：`if (strlen(a) > strlen(b))` 会出错——因为 `strlen` 的返回类型是**无符号数**，两个无符号数相减永远不会小于 0。

正确做法是先存入有符号变量：

```
int len1 = strlen(arr1);
int len2 = strlen(arr2);
if (len1 > len2) { /* ... */ }
```

- 字符串处理中涉及「删除空格、删除指定字符、删除相同字符」的题目，**核心思路不变**，只是判断语句不同——先理清“要保留什么”，再写条件。

易错

易错：`strlen(a) - strlen(b) > 0` 恒为真，这是 C 语言最经典的陷阱之一。

22 典型易错题演算

```
int a, b, c, t;
a = 1; b = 2; c = 2;
while (a < b < c) {
    t = a; a = b; b = c; c--;
}
cout << a << ", " << b << ", " << c << endl;
```

结果：`1,2,0`，演算过程：

- 判断 `a < b < c`：先算 `a < b` 即 `1 < 2` 得 **1**（真），再算 `1 < c` 即 `1 < 2` 得 **1**，条件为真，进入循环体。
- 执行 `t = a` (`t=1`)、`a = b` (`a=2`)、`b = c` (`b=2`)、`c--` (`c=1`)。
- 再次判断 `a < b < c`：`2 < 2` 得 **0**，`0 < 1` 得 **1**，条件仍为真！进入循环体。
- 执行 `t = a` (`t=2`)、`a = b` (`a=2`)、`b = c` (`b=1`)、`c--` (`c=0`)。
- 再次判断：`2 < 1` 得 **0**，`0 < 0` 得 **0**，条件为假，退出循环。
- 最终 `a = 2`？——注意第 4 步 `b = c` 执行时 `c` 还是 1，随后 `c--` 使 `c` 变为 0；此时 `a = 2, b = 1, c = 0`。

易错

易错：`a < b < c` 不是「a 小于 b 且 b 小于 c」，而是「(a<b) 的结果再与 c 比较」。这类题必须**一击一算**，不能心算跳步。

23 一条通用检查清单

考前做题时按以下顺序自查，可覆盖本章绝大多数陷阱：

- 类型**：这个表达式的类型是什么？会不会发生隐式转换？赋值后变量类型变了吗？
- 符号**：`const` 修饰的是指针还是值？`*` 是定义还是解引用？`=` 还是 `==`？
- 优先级**：`a < b < c`、`!m % i`、`&&` 与 `=` 混用，是否需要加括号？
- 边界**：数组下标是否越界？循环是否会死循环？字符串是否算了 `'\0'`？

5. **对象**：静态还是非静态？有没有 `this`？构造与析构的顺序对不对？
6. **内存**：堆区数据有没有配对 `new` / `delete`？有没有浅拷贝导致的重复释放？

结语：复习方法与自测清单

1 一、三遍复习法

遍次	目标	做法
第一遍：通读	建立全局框架	只看每章标题与「知识清单」，不写代码。目标是能画出全书的知识地图
第二遍：动手	把知识变成能力	每章至少手写 2 个完整程序并编译运行，重点：构造/析构、深浅拷贝、运算符重载、继承、多态、模板、STL 容器
第三遍：应试	查漏补缺	只看第 24 章应试重点与第 25 章易错清单，配合历年题限时练习

技巧

技巧：第二遍时把编译报错信息抄下来。**报错信息本身就是最好的易错清单**，比任何总结都贴近你的真实薄弱点。

2 二、按题型准备

题型	考查重点	准备方式
选择题	概念辨析、语法细节	扫第 24 章与第 25 章的结论句；注意「唯一区别」「不能」「必须」这类绝对表述
判断题	细节真伪	关注 <code>const</code> 位置、静态/非静态、默认权限、字符串长度、 <code>strlen</code> 无符号等高频陷阱
填空/简答	术语与语法格式	背 准 确 的 语 法 格 式： <code>template<class T></code> 、 <code>delete[]</code> 、 <code>virtual ~类名() = 0;</code> 等
程序阅读题	执行结果推演	在纸上逐行演算变量表；特别注意自增前后置、 <code>a<b<c</code> 、循环退出条件
程序改错题	找错与修正	对照第 25 章的通用检查清单（类型、符号、优先级、边界、对象、内存）逐项排查
编程题	综合运用	熟练「数组 + 函数 + 指针/引用」「类 + 构造析构 + 运算符重载」「继承 + 多态 + 虚析构」三类模板写法

3 三、核心自测清单

能在不看资料的情况下准确回答下列问题，说明本课程的核心知识已经掌握。

● 3.1 语法基础

1. `sizeof` 与 `strlen` 的区别？`"abc"` 占几个字节？
2. 前置 `++` 与后置 `++` 在表达式中的差别？
3. `do...while` 与 `while` 最本质的区别是什么？
4. 数组名是什么？`sizeof(arr)` 与作为函数参数时的 `sizeof(arr)` 结果为什么不同？
5. 值传递、地址传递、引用传递三者的区别与选择准则？
6. 一级指针的三种 `const` 修饰分别限制了什么？
7. 结构体变量与结构体指针访问成员分别用什么运算符？

● 3.2 面向对象

1. `struct` 与 `class` 的唯一区别是什么？
2. 构造函数和析构函数的四个特点分别是什么？
3. 拷贝构造函数在什么时机被调用？「拷贝构造」与「赋值」的差别？
4. 深拷贝与浅拷贝的区别，为什么浅拷贝会导致程序崩溃？
5. 静态成员变量为什么必须类内声明、类外初始化？静态成员函数为什么不能访问非静态成员？
6. `this` 指针的两个典型用途？
7. 友元有哪三种形式？友元破坏封装性吗？
8. 前置 `++` 与后置 `++` 重载的返回值为什么一个是引用一个是值？
9. 三种继承方式对基类成员权限的影响？默认继承方式是什么？
10. 继承中构造与析构的调用顺序？
11. 多态的三个条件是什么？重写的定义是什么？
12. 为什么析构函数通常要声明为虚函数？
13. 纯虚函数与抽象类的关系？

● 3.3 提高篇

1. 函数模板的两种使用方式？模板具体化解决什么问题？
2. 类模板为什么没有自动类型推导？类模板成员函数何时被创建？
3. 类模板为什么分文件编写会链接失败？两种解决办法是什么？
4. `vector` 的 `resize` 与 `reserve` 的区别？
5. `vector`、`deque`、`list` 在插入删除与随机访问上的差异？
6. `set` 与 `multiset`、`map` 与 `multimap` 的区别？
7. 为什么 `list` 必须用成员函数 `sort()`？

- 8. 仿函数与谓词的定义？内建仿函数在哪个头文件中？
- 9. 哪些算法要求容器**必须有序**？哪些算法要求目标容器**提前** `resize`？
- 10. 集合算法 `set_intersection` 的返回值为什么要作为遍历的结束位置？

4 四、常见失分点提醒

- **凭感觉答题**：程序阅读题必须动笔演算，尤其是循环与自增混合的题目。
- **忽略边界**：数组下标从 0 开始；字符串长度不算 `'\0'`；循环要从条件为假时退出。
- **混淆符号**：`=` 与 `==`、`.` 与 `->`、`*` 的声明与解引用含义、`const` 的位置。
- **忘记释放**：`new` 必须有配对的 `delete`；`new[]` 必须配 `delete[]`；有堆区成员的类型要写深拷贝与虚析构。
- **不做题**：知识点看懂了不等于会做题。应试能力最终靠**练**出来。

说明

最后提醒：本书是对六份课程资料的系统整理，其中的结论以 C++ 标准语义与教材原文为准；个别思维导图识别条目已按标准语义校正。若与任课老师课堂强调的内容不一致，请以课堂讲授与所用教材为准。

祝复习顺利、考试顺利。